MicroEMACS:
Reasonable Editing
on
Tiny Computers

by
David G. Conroy
Digital Equipment Corporation
146 Main St.
Maynard, MA 01754

revised by
Mark Alexander
marka@pobox.com
formerly at:
Digital Research, Inc.
70 Garden Court
Box DRI
Monterey, CA 93942

October 12, 2026

Contents

1 Introduction **4**
1.1 History . . . 4
1.1.1 2018 Update . . . 4
1.1.2 Current Source Tree . . . 4

2 Some Basic Concepts **4**
2.1 Screen Layout . . . 5
2.2 Mode Lines . . . 5
2.3 Keyboard Conventions . . . 5
2.3.1 The PC Keyboard . . . 6
2.3.2 The Zenith Z-19/29 Keyboard . . . 6
2.3.3 The VT-100 Keyboard . . . 6
2.3.4 The LK201 Keyboard . . . 7
2.4 Key Bindings . . . 7
2.5 The Echo Line . . . 7
2.6 Command Arguments . . . 8
2.7 Dot, Mark, and the Region . . . 8

3 Starting **8**

4 Quitting **10**

5 Moving Around **11**

6 Inserting **13**

7 Deleting and Killing **15**

8 Rectangles **16**

9 Words **17**

10 Paragraphs **17**

11 Case Conversion **18**

12 Searching and Spelling **19**

13 Files **21**

14 Hexl Editing **23**

15 Keyboard Macros **24**

16 Buffer Management **24**

17 Window Management **25**

18 Messages **27**

19 Key Binding Commands **27**

20 Tags **28**

21 Error navigation **29**

22 Undo **29**

23 Profiles **29**

24 Lisp Extensions **32**

25 Display Features **35**
25.1 Colors . . . 35
25.2 Syntax Highlighting . . . 36
25.3 Line Numbers . . . 36

26 UTF-8 and Unicode **37**

27 Building a MicroEMACS **37**
27.1 Operating System . . . 37
27.2 Terminal Support . . . 37
27.3 Building with build.sh . . . 38
27.4 Windows Build . . . 38
27.5 Direct Make Builds . . . 38

28 Wall Chart **40**

1 Introduction

This document describes MicroEMACS, a public-domain[1] display editor, loosely based on EMACS, that runs on little computers. It is intended as a reference manual for users already familiar with EMACS.

We call it Micro *EMACS* to emphasize the fact that most of the commands are very similar to, if not identical to, fundamental mode EMACS commands (actually, it implements a set of rebindings used by a group of users at Digital Equipment Corporation[2]).

We call it *Micro* EMACS to emphasize the fact that it is but a shadow of full EMACS. No attempt has been made to make MicroEMACS wildly customizable (without writing code), or to have extensive online documentation. All of this sophistication was thrown away right at the start, because it was important that MicroEMACS run on little computers. In all fairness, is should be stated here and now that the most popular small computer these days is the MicroVAX[3]!

1.1 History

MicroEMACS is loosely based on the EMACS display editor written by Richard Stallman at MIT. The MicroEMACS described by this document is Conroy's version 30 of February 1986, distributed to USENET mod.sources. Since then it has undergone a fair number of bug fixes and performance improvements, and a few minor enhancements.

This version of MicroEMACS is not to be confused with other popular versions that are derived from Conroy's November 1985 release. These other versions of MicroEMACS have version numbers like 3.8 or 3.9, are considerably larger and feature-rich, are copyrighted, and are now being maintained by Daniel Lawrence.

I (Mark Alexander) converted this document from TeX to Scribe to FinalWord 1.15 to Borland Sprint, and finally to Pandoc/Latex, since Sprint is no longer available and doesn't run on Linux. I added new material to bring the document up to date with the program itself.

1.1.1 2018 Update

I have lost the original source of Conroy's version of MicroEMACS (including the TeX version of this document), and I cannot find it on Google's USENET archive. I have also lost my DOS and OS/2 source code. This document contains many historical references to machines and operating systems, but I have kept them for historical interest. The current source tree builds both Linux and Windows versions.

1.1.2 Current Source Tree

This source tree currently builds a Linux executable with ncursesw or termcap, and a Windows executable with MinGW-w64. It includes UTF-8 editing, syntax highlighting, optional colors, optional line numbers, tags, cscope, undo, and regular-expression search and replace.

In the 80s, MicroEMACS was small enough to run easily from a floppy disk, but the amount of text that could be edited was limited by the very small amount of available RAM (640kb on PCs running MS-DOS). Nowadays this limit is effectly non-existent, give the huge amount of memory found in modern computers, but MicroEMACS is still small enough to be run from a floppy disk (if one could be found); its code size is about the same as `/bin/ls` on a current 64-bit Linux distribution. The current Linux and Windows executables remain small while including these features.

2 Some Basic Concepts

This section describes a number of basic concepts, which will be used again and again in the following descriptions. Most of it should be familiar to EMACS users. New users are advised not to get stuck reading this section, because most of the information will become clear the first time you play with the editor on a computer.

[1] The source is now copylefted with the GPL.
[2] Further modified at DRI to more closely resemble, but not completely emulate, MINCE..
[3] This appears to be a reference by Conroy to DEC's attempt to ignore the IBM PC.

2.1 Screen Layout

MicroEMACS divides the screen into two areas. The very last line on the screen, which is used to display messages and ask one line questions, is called the *echo line* . Typing in the echo line is like typing commands to the operating system. More details later in the section **The Echo Line**.

The remainder of the screen is filled with text *windows* . When MicroEMACS is first started, there will be one or two text windows, depending on how many files you specify on the invocation line. If there is only one window, it fills the screen. If there are two windows, each occupies half the screen. Windows can be created, deleted, and adjusted in size using MicroEMACS commands.

Line numbers can be displayed in a narrow left gutter by starting MicroEMACS with the `-n` option, or by using the **set-number** command. The gutter is only as wide as the largest line number needs, and one space separates the number from the text.

2.2 Mode Lines

The last line of a window (the line, usually in reverse video, that starts with `MicroEMACS`) is the *mode line* for the window. It contains information about what is being displayed in the window. Mode lines separate windows from each other and from the echo line.

Four important bits of information are displayed in the mode line. These are the buffer *mode* (if any) in parentheses, the *buffer name* , the *file name* , and the *buffer changed flag* .

A window is always displaying a buffer. The name of the buffer being displayed in the window is shown in the mode line, right after the `MicroEMACS` and the buffer mode.

A buffer may have a file associated with it. Usually this is the last file read into or written from the buffer. If a file is associated with the buffer, the name of the file is displayed in the mode line, prefixed by `File:`. The file name is shown exactly as it is stored with the buffer; it is not normalized, converted to a relative path, or made relative to the current working directory. On operating systems in which file names are case insensitive (for example, VMS) the file name will always be in lower case.

MicroEMACS knows if a buffer has been changed since it was last written to its associated file. It informs you of this fact by displaying a `*` in the mode line immediately to the left of the `MicroEMACS` string.

2.3 Keyboard Conventions

Internally, MicroEMACS represents keyboard characters in a special 11 bit code. This code consists of 8 bits of character data (ASCII plus whatever characters your terminal puts in the right side graphics set), and 3 flags, the *CONTROL* flag, the *META* flag, and the *CTLX* flag. Combinations of the three flags are possible, with the exception that META and CTLX cannot be set at the same time.

This manual represents 11 bit characters in a standard way. If you have a character `A`, then that character with the *CONTROL* bit set is written as **C-A**. That character with the *META* bit set is written as **M-A**. That character with the *CTLX* bit set is written as **C-X A**. If both the META and CONTROL bits are set, it is written as **M-C-A**. If both the CTLX and CONTROL bits are set, it is written as **C-X C-A**.

Terminals (usually) cannot deal directly deal with the 11 bit character set. Characters with the flag bits set are entered by prefixing the desired character with another character, as follows:

- To get a character with the *CONTROL* bit set, hold down the `CTRL` key while entering the character. An alternative method is to prefix the character with `Control-^` (the word `Control` here, and in the next few examples, means the key marked `Control` or `CTRL` on the ASCII keyboard).
- To get a character with the *META* bit set, prefix the character with `Control-[`, `Escape`, or `ESC`. An alternative method that works on FlexOS is to hold down the `ALT` key while entering the character. This method also works on PC-DOS, but only with `ALT-A` through `ALT-Z`.
- To get a character with the *CTLX* bit set, prefix the character with `Control-X`.
- To get a character with the *CONTROL* and *META* bits set, prefix the character with `Control-\`. An alternative method is to prefix the character with `ESC`, then hold down `CTRL` while entering the character.

- Those of you who understand ASCII are probably asking what you get if you prefix a character with `Control-]`. This character is not a prefix; it is reserved for future use as an "exit from recursive editing level" command.

CONTROL characters are very important, so there is a shorthand way of typing them, as follows:

- Characters between `Control-@` (`Control-Space` on some keyboards) and `Control-Z` are mapped to the characters `@` through `Z` with the *CONTROL* bit set.
- Characters between `Control-a` and `Control-z` are mapped to the characters `A` through `Z` with the *CONTROL* bit set.

This shorthand means that that **M-C-V**, for example, can be entered either as `ESC` followed by `Control-V`, or as `Control-\` followed by `V`. As another example, **C-X C-B** is entered as `Control-X` followed by `Control-B`.

2.3.1 The PC Keyboard

MicroEMACS understands all of the cursor keypad and function keys on PC compatible machines, and the equivalent keys on other machines that run FlexOS (such as the VME/10). These keys include the function keys (F1-F10), shifted function keys (S-F1 to S-F10), cursor keys (Up, Down, Left, Right, PgUp, PgDn, Home, End), Delete, and Insert. These keys may be combined with the ALT and CTRL keys.

On PCs, you can use the `ALT` key as the META flag, instead of the `ESC` key. For example, the **M-L** command (lower-word) can be executed in either of two ways:

- Hit `ESC` followed by `L`.
- Hold down `ALT` and hit `L`.

On PC-DOS and Linux, the `ALT` key only works with alphabetic characters (`A` through `Z`). Using `ALT` and `CTRL` together is also not supported.

On FlexOS, the `ALT` key works with both alpha and non-alpha characters, and simultaneously with the `CTRL` key. Thus, the command **M-C-V** (scroll-next-down) can be executed by holding down `ALT` and `CTRL`, while pressing `V` (not an easy task, in this writer's opinion).

On the VME/10, some of the cursor keys are not available, due to the inherent physical limitations of the keyboard itself.

See the **Wall Chart** section of this manual for the predefined bindings of some the special PC keys.

2.3.2 The Zenith Z-19/29 Keyboard

MicroEMACS treats the numeric keypad on the Z-19/29 similarly to the EDT/VMS customizations in common use at DRI. It does this by putting the terminal in "alternate keypad mode", so that the keypad produces escape sequences instead of numbers. The arrow keys (2, 4, 6 and 8) move one line or one character at a time; when shifted they move by pages or word. When preceded by the F6 key (BLUE on Z-19), the arrow keys move to the beginning or end of the current line or buffer. The 5 key is the same as **C-S** (forw-i-search), and shift-5 is search-again. See the **Wall Chart** section of this manual for further key bindings for the Z-19/29.

Not all of the key bindings that allow the shifted arrows to work like EDT are built into MicroEMACS. This is because they conflict with the traditional definitions of **M-B** and **M-C**. Therefore, these rebindings are placed in a profile called **edt.pro**. If you want MicroEMACS to read these bindings when it starts up, and you don't care about losing the old meanings of **M-B** and **M-C**, then copy **edt.pro** to the default profile name, as described below in the **Profiles** section.

2.3.3 The VT-100 Keyboard

MicroEMACS understands the arrow keys and the keypad on VT-100 style keyboards. Only the PF1 through PF4 keys on the keypad have any special meaning; the digits, the `-` key, and the `,` key are treated just like the non keypad versions of those characters. The `Enter` key is treated exactly like the `Return` key.

The arrow keys may be used for moving around. The left arrow is the same as **C-B**, the right arrow is the same as **C-F**, the up arrow is the same as **C-P**, and the down arrow is the same as **C-N**.

The four function keys are command keys. The `PF1` key is the same as **M-X** (the `PF1` key is where the `Gold` key normally resides on DEC products, so it is a good choice for the key that asks for extended commands). The `PF2` key is the same as **C-Q** (quote character). The `PF3` key is the same as **C-S** (incremental search forward). The `PF4` key is the same as **C-R** (incremental search reverse). These assignments have not been proven optimal, so they may get changed in the future.

2.3.4 The LK201 Keyboard

The escape key, used in all *META* commands, is very poorly placed on the LK201 keyboard. To make typing a bit easier, the grave accent is also considered to be a *META* prefix on the LK201; the grave accent is right where you expect the escape key to be located. A grave accent must be quoted to exter it into the text.

The arrow keys and all of the keys on the keypad work just like they do on the VT-100 keyboard.

The keys immediately above the arrow keys try to do what their name implies. `Next Screen` is the same as **C-V**. `Prev Screen` is the same as **M-V**. `Select` is the same as **C-@** (set mark). `Remove` is the same as **C-W** (kill region). `Insert here` is the same as **C-Y** (yank from killbuffer). `Find` is the same as **C-S** (incremental search forward).

The `F6`, `F7`, `F8`, `F9`, `F10`, and `F14` keys are unused. The `F11` is escape, which is a *META* prefix key. The `F12` key is backspace. The `F13` key is linefeed.

The `Help` key is unused. The `Do` key is the same as **C-X E** (execute keyboard macro).

The `F17` key is the same as **C-X P** (previous window). The `F18` key is the same as **C-X N** (next window). The `F19` key is the same as **C-X Z** (grow window). The `F20` key is the same as **C-X C-Z** (shrink window).

2.4 Key Bindings

Normally when you type a key, MicroEMACS attempts to interprets the key as a command. Internally, MicroEMACS keeps a table of each possible key, and the command assocated with that key. This association is called the "key binding."
Even text keys, such as the letters, numbers, and punctuation, are "bound" to a command called "ins-self" that just inserts the key that invoked it into the text.

Not all commands are bound to a particular key. These commands can still be entered by using the (**M-X**) (`ESC X`) "extended-command" prefix, or by binding them to a key with the "bind-to-key" command. See the section **Key Binding Commands** below for more information.

In subsequent sections, the command descriptions will give not only the key that the command is normally bound to, but also the command name, which is useful when you want to change the key bindings.

2.5 The Echo Line

The echo line has two purposes; displaying messages and asking one line questions.

Two types of messages are displayed in the echo line. Informational messages tell you something useful, but do not imply in any way that something is wrong. These messages are always enclosed in square brackets ([and]). Error messages indicate that something has prevented a command from being executed. They are never enclosed in square brackets, and will be accompanied by a beep from the terminal's bell if the sound of the bell is not too obnoxious.

The echo line is cleared by the next keystroke.

The echo line is also used for asking and answering questions. After the prompt, you can type any characters you like. The reply is always terminated by a `Return`. Before you commit to the reply, you can edit the reply using the following characters:

- `Backspace` or `Rubout`: delete the character to the left of the cursor.
- `Control-A`: move the cursor to the beginning of the line.
- `Control-B`: move the cursor one character to the left.
- `Control-D`: delete the character under the cursor.
- `Control-E`: move the cursor to end of the line.

- `Control-F`: move the cursor one character to the right.
- `Control-K`: delete from the cursor to the end of the line.
- `Control-Y`: insert the first line of the kill buffer at the cursor.
- `Control-Q`: enter the next character literally into the line (useful for entering control characters).
- `Control-U`: delete the entire line.

You can also abort the command in progress by typing `Control-G`. Command processors are designed to ask all questions before doing nasty things, so that you will never do damage by aborting a command.

The echo line supports autocompletion. If you are entering a command name, a filename, or a buffer name, you can use the `Space` or `Tab` keys to tell MicroEMACS to complete as much of the entry as possible, based on what you have entered so far. If you are entering a filename, pressing `Control-S` will fill in the directory part of the current buffer's filename, making it easier to find a file in the same directory. Pressing the `?` or `Control-D` keys will open a new temporary window containing the possible list of choices.

If you are entering a search string, pressing `Control-S` will fill in the previous search string.

2.6 Command Arguments

All commands can be given a numeric argument. Most commands use this argument as a repeat count. Commands can tell if an argument has been supplied, and will use a default argument if no user argument is present. Usually, the default argument is `1`.

A **C-U** preceding a command always introduces an argument.

As an alternative, **M-0** through **M-9** start a decimal argument, and **M**– starts a negative decimal argument. Additional digits are appended until the next command key is entered. For example, **M-3 M-C-F** moves forward three characters, and **M– 2 C-N** moves back two lines.

If a numeric argument is just a string of **C-U** characters, then the value of the argument is 4^(number of **C-U** characters), where "^" means "to the power." Therefore **C-U** is 4, **C-U C-U** is 16, **C-U C-U C-U** is 256, and so on. Some commands care if an argument is present, but don't look at its value; the **C-U** character is a good way to say "here is an argument".

If there is a signed decimal number after the string of **C-U** characters it specifies the value of the argument. In this case the number of **C-U** characters does not matter. An argument of 10 can be represented by **C-U 10**, **C-U C-U 10**, or even **C-U C-U C-U C-U 10**.

2.7 Dot, Mark, and the Region

Most editing commands act on the character or characters surrounding the current location in the current buffer. The thing that marks the current location is always called *dot* . The dot always points between two characters. This isn't the way that the cursor works on most display terminals, so, by convention, dot is always immediately to the left of the character on which the hardware cursor is placed.

There is also a second remembered position in the buffer, called the *mark* . There are commands that set the value of mark.

Some commands act on a block of text called the *region* . The region is all of the characters between dot and mark. The relative positions of dot and mark do not matter.

3 Starting

Start editing by typing the command

```
me [-234bCDHmnNqrTuxz] [-c path] [-d path] [-g line]
   [-l lisp] [--script lisp] [-p profile]
   [-s string] [-t size] [+line]
   file[:line[:column]] ...
```

The options are optional. They have the following meanings:

-b Tells MicroEMACS to make backups of the files you edit. It will create a backup file the first time you save a file that has been changed. It makes the backup by renaming the original file before it writes out the new version. If you have defined an XBACKUP environment variable, MicroEMACS treats it as a directory name and moves the original file into that directory. If you haven't defined XBACKUP, MicroEMACS renames the original file in one of two ways:

- On UNIX-like operating systems, such as Linux,, it appends a "~" character to the filename.
- On PC-DOS and FlexOS, it changes the extension to ".BAK".

This option has no effect on VMS, because VMS supports multiple file versions.

-2, -3, -4 Divides the terminal into that number of side-by-side pages.

-C Disables color output in builds compiled with color support. Color is enabled by default in the current Linux and MinGW builds.

-c *cscope-path* Tells MicroEMACS the path to the cscope executable, useful if you are using a non-standard cscope.

-d *directory* Changes to the specified directory before opening files.

-D Uses DOS/CRLF line endings for newly created buffers. This is useful when editing files for Windows software that requires CRLF endings.

-H Opens the files named on the command line as hex dumps, as the **Hexl Editing** section describes, instead of as text.

-u Tells MicroEMACS to pass the `-d` flag to cscope, which tells it to not update its cross-reference file (`cscope.out`). This is useful when using a cross-reference file generated by another program, such as starscope.

-g *linenum*, +*linenum* Tells MicroEMACS to move to the specified line number in the first file specified on the command line. The `+line` form is provided for compatibility with other editors.

-l *lisp* Tells MicroEMACS to load the specified Lisp startup file instead of the default Lisp startup file. See the **Lisp Extensions** section for more information.

-m Tells MicroEMACS to enable the experimental mouse support: click to move point, double click to select the word under the mouse (a third click selects the whole line), drag to select the mark-to-point region, a shifted or control drag to sweep out a rectangle (anchored at the pressed cell and left set for the C-X r commands; control for the terminals, st and xterm among them, that keep a shifted drag for their own text selection), a middle click to yank the kill buffer at the clicked cell, a right click to extend the region — or the anchored rectangle, when one is set, and a plain click then drops the anchor — and the wheel to scroll the window under the mouse, by `mouse-wheel-step` lines per tick (a shifted wheel scrolls horizontally; terminals like st that send the Shift+PageUp/PageDown keys for it scroll the window under the cursor instead). Reports are decoded in both the SGR and the legacy X10 encoding, so terminals without SGR mouse support still work. Inside a multiplexer, turn its own mouse handling off so the reports reach the editor: tmux `set -g mouse off`, GNU screen `mousetrack off`. dvtm needs nothing: when the editor enables mouse tracking it arms reporting on the outer terminal itself and forwards the reports, in the client window's coordinates, to the editor. Warning: this is not fully functional. This option is available only in builds compiled with `-DMOUSE`; the `toggle-mouse` extended command can change it while the editor is running — a numeric argument of zero turns reporting off, a nonzero one turns it on, and without one the setting is toggled.

-n Displays line numbers in each text window. The **set-number** command can also toggle line numbers while MicroEMACS is running.

-N Disables the automatic newline added at the end of a file when it is saved.

-s *string* Moves to the first occurrence of `string` after the startup files are loaded.

-T Expands tabs into spaces when the buffer is saved.

-t *size* Sets the tab size. Valid sizes are 2 through 32; invalid values fall back to the default size of 8.

-x Tells MicroEMACS to use the XON/XOFF protocol for starting and stopping transmission to your terminal. This is necessary if you are using a DEC terminal, or any terminal that might lose characters during lengthy operations, such as multi-line scrolls. This option is probably not necessary for Zenith Z-29 terminals, or if your computer uses a memory-mapped display. Unfortunately, this option prevents you from using Control-S and Control-Q while editing. You can get around this problem by reassigning the functions that normally are invoked with these keys to other keys. These key assignments could be placed in your startup profile (see the **Profiles** section below).

-z Tells MicroEMACS to append a Control-Z to output files, and to treat Control-Z as an end-of-file character when reading text files or profiles. This option is provided for compatibility with older editors and language processors. It has no effect on VMS or UNIX.

MicroEMACS also accepts `--help` and `--version` when either is the only command-line argument. `--help` prints a grouped summary of command-line options to standard output; unavailable options are omitted in builds that do not include their features. Minimal builds use the compact legacy usage.

-p *profile* Tells MicroEMACS to read the specified *profile* at startup, instead of the default profile. See the **Profiles** section for more information about profiles.

-q Skips the default startup profile and the default Lisp startup file. Explicit **-p** and **-l** files are still loaded.

-r Tells MicroEMACS to mark all buffers as read-only by default. This is useful in preventing unwanted changes being made when browsing files. A buffer can subsequently be made read-write with the **toggle-readonly** command.

The filenames *filename1*, *filename2*, etc., are the names of the files you want to edit. MicroEMACS will load the specified files into separate buffers, and you can start editing them.

You can append a line number and an optional column number to a filename. For example, `me main.c:168:27` opens `main.c` and moves to line 168, column 27. The `file://main.c:168:27` URI form is also accepted.

MicroEMACS will create up to two split-screen windows to view the first two files you specify. If you specify only one file, MicroEMACS will create one full-screen window. The commands for manipulating windows are described in the **Window Management** section below.

Any changes you make to a file while it is in a buffer will not affect the original file until you tell MicroEMACS to save the file.

You can omit the filenames when you start. MicroEMACS will present you with an empty buffer called “main”, which has no associated filename. You can give this buffer a filename, or read a file into it, or you can read files into their own separate buffers. The commands for reading and saving files are described in the **Files** section below.

After MicroEMACS reads in the files you specify (if any), but before it accepts any keyboard entries, it executes the commands in the startup profile (either the one you specified with the -p option, or the default profile).

4 Quitting

When you finish editing, you have to quit. MicroEMACS never writes out a file because it thinks that this is the right thing to do. However, it will attempt to protect you from forgetting to do so.

C-X C-C **quit**

This is the basic quit command. MicroEMACS exits, returning control to the shell. If there are any changed buffers that have not been written out, it will ask for permission to quit. Supplying an argument to **C-X C-C** makes the command quit unconditionally, without asking for confirmation. The value of the argument is not important. On PCs, this function is also bound to `F4`.

M-Z **quick-quit**

Save every changed buffer that has an associated file, then quit. If a changed buffer has no associated file, MicroEMACS still asks for confirmation before quitting. An argument is passed to each save and to the final quit.

C-X C **spawn-cli**

This command suspends the execution of MicroEMACS, and runs a command interpreter in a subjob. When the subjob terminates, the screen is cleared and repainted.

Subjobs are implemented in FlexOS, PC-DOS VMS, and UNIX-like operating systems, such as Linux. Users of CP/M are out of luck.

Exit the command interpreter and return to MicroEMACS with one of the following:

- On VMS enter the **logout** command.
- If you are using the c-shell or bash on Unix-like operating systems, enter the **fg** command.
- On all other systems enter the **exit** command.

C-X ! **shell-command**

Prompt for a shell command, run it in a subjob, and wait for a keypress at `(End)`. MicroEMACS then repaints the screen.

[unbound] **jeff-exit**

This is a slightly more elaborate quit command. If the current buffer has been changed and is not the `main` buffer, it saves the contents of the buffer in the associated file. Otherwise it acts like **spawn-cli**.

5 Moving Around

C-@,C-space,M-space set-mark

Set the value of the mark to be equal to the dot. Use this command to mark the beginning of a region of text to be operated on by a region command such as **C-W**. You can also use this command to mark current location so it can be returned to later by **C-X C-X**. This command also pushes the new value of the mark onto a 16-entry ring of marks for the current window.

If you precede this command with an argument (i.e., precede it with a **C-U**), it sets the dot to the most recent mark that was stored in the ring, and moves that mark to the end of the ring. It also sets the mark to the next most recent mark in the ring, if any. By repeatedly using this command with an argument, you can cycle through the entire ring of marks, returning to those marked locations in sequence.

[unbound] **clear-mark**

Clear the mark for the current window. This is useful when you no longer want region commands to operate on the previous dot-and-mark region.

C-X C-X **swap-dot-and-mark**

Exchange the positions of the dot and the mark. This is useful for switching back and forth between two points in a file.

C-X H **mark-all**

Put dot at the beginning of the buffer and set the mark at the end of the buffer, selecting the entire buffer as the region.

C-A **goto-bol**

Move to the beginning of the current line. Any argument is ignored. Always succeeds. On PCs, this function is bound to the `Home` key.

C-B **back-char**

Move backwards character by character. The number of characters to move is specified by the argument. If no argument is given it moves backwards by 1 character. A newline counts as a single character. Fails if executed at the beginning of the buffer. On PCs, this function is bound to the `Left` arrow key.

C-E **goto-eol**

Move to the end of the current line. Any argument is ignored. Always succeeds. On PCs, this function is bound to the `End` key.

C-F **forw-char**

Move forwards character by character. The number of characters to move is specified by the argument. If no argument is given it moves forwards by 1 character. A newline counts as a single character. Fails if executed at the end of the buffer. On PCs, this function is bound to the `Right` arrow key.

C-N **forw-line**

Move forward by lines. Attempt to preserve the current horizontal position. The number of lines to move is specified by the argument. If no argument is given it moves by 1 line. Fails if executed at the end of the buffer. On PCs, this function is bound to the `Down` arrow key.

C-P **back-line**

Move backwards by lines. Attempt to preserve the current horizontal position. The number of lines to move is specified by the argument. If no argument is given it moves by 1 line. Fails if executed at the beginning of the buffer. On PCs, this function is bound to the `Up` arrow key.

C-V **forw-page**

Move forward by pages. If an argument is given, it specifies the number of pages to move. If no argument is given, 1 is assumed. A page is a group of lines about 20% smaller than a window. If possible, dot is kept where it is; otherwise it it moved to the middle of the new page. On PCs, this function is bound to the `PgDn` key.

There is a compile time option that makes this command take an argument in lines instead of screenfuls. Look in `def.h` for the gory details.

M-V,C-Z **back-page**

Move backwards by pages. If an argument is given, it specifies the number of pages to move. If no argument is given, 1 is assumed. A page is a group of lines about 20% smaller than a window. If possible, dot is kept where it is; otherwise it it moved to the middle of the new page. On PCs, this function is bound to the `PgUp` key.

There is a compile time option that makes this command take an argument in lines instead of screenfuls. Look in `def.h` for the gory details.

M-< **goto-bob**

Move to the beginning of the buffer. Any argument is ignored. Dot is set to the first character of the first line in the buffer. On PCs, this function is bound to the `Control-Home` key.

M-> **goto-eob**

Move to the end of the buffer. Any argument is ignored. Dot is set to the first character in the fake line immediately after the buffer. The window is set to display dot near the center of the screen. On PCs, this function is bound to the `Control-End` key.

C-X G **goto-line**

This command moves the dot to a specific line number. If an argument is provided, the dot is moved to that line number. If no argument is provided, the command prompts on the echo line for a line number. This is useful for fixing a source file using error messages produced by a compiler.

C-X = **display-position**

This command displays information about the current position of the dot. The information is displayed on the echo line, and includes the following:

- The octal and hex values of the character at the dot.
- The current line number, in decimal.
- The current screen row and column, in decimal.
- The approximate position of the dot in the buffer, measured as a percentage of the buffer size.
- The number of characters in the buffer.
- Whether the current buffer uses **LF** or **CRLF** line endings.

6 Inserting

All characters between hexadecimal 20 and 7E (blank through tilde), and all characters between hexadecimal A0 and FE (right hand graphics) are self-inserting. They are inserted into the buffer at the current location of dot, and dot moves 1 character to the right.

The **Tab** (or **C-I**) key is also self-inserting. By default, MicroEMACS has fixed tab settings at columns 9, 17, 25, and so on, but you can change the tab width using the **set-tab-size** command.

Any self-inserting character can be given an argument. This argument is used as a repeat count, and the character is inserted that number of times. This is useful for creating lines of `*` characters of a specific length, and other pseudo-graphic things.

Self-inserting keys are all bound to the function **ins-self**. As with any key, these keys can be rebound to other functions, but this is not always advisable.

C-C **ins-space**

Insert spaces at dot. The argument specifies the number of spaces. Dot remains at the first inserted space, which makes this command useful for leaving room for later text.

Word wrapping can be performed by binding a key to the **ins-self-with-wrap** function. This function is described in the **Paragraphs** section below.

C-M, Return **ins-nl**

The Return key works just like you would expect it to work; it inserts a newline character. Lines can be split by moving into the middle of the line, and inserting a newline.

On some terminals with slow displays, if dot is positioned at end of line, and the line after the current line is a blank line, then **C-M** does not insert a newline, but simply moves to the first position on the following line. This lets you create a block of blank space (perhaps using **C-O** and then type text into it. This "feature" can be enabled at compile-time by setting the "NLMOVE" definition to 1 in `def.h`.

[**unbound**] **ins-nl-and-indent**

This command is like a **C-M** with indentation. It inserts a new line, and then inserts enough tabs and spaces to duplicate the indentation of the previous line. This is useful for entering heavily-indented programs in structured languages like PASCAL or C.

[**unbound**] **borland-indent**

This command is like a **ins-nl-and-indent**, but also attempts to indent according to the coding standards in use at Borland in the 1990s. If the previous line starts with `{`, or an argument of four (i.e. a single `Control-U`) is specified, indent by four spaces. If an argument of 16 (i.e. two `Control-U`s) is specified, reduce indentation by four spaces. Otherwise retain the same indentation.

C-J **gnu-indent**

This command is like a **ins-nl-and-indent**, but also attempts to indent according to the GNU coding standards in use at Cygnus in the 1990s. If the previous line starts with `{`, `if`, `while`, `for`, `else`, `case, or an argument of four (i.e. a single`Control-U`) is specified, indent by two spaces. If an argument of 16 (i.e. two`Control-U's) is specified, reduce indentation by two spaces. Otherwise retain the same indentation.

[**unbound**] **vmware-indent**

This command is like a **ins-nl-and-indent**, but also attempts to indent according to the coding standards in use at VMware in the 2000s. If the previous line starts with `{`, or an argument of four (i.e. a single `Control-U`) is specified, indent by three spaces. If an argument of 16 (i.e. two `Control-U`s) is specified, reduce indentation by three spaces. Otherwise retain the same indentation.

C-O **ins-nl-and-backup**

This command creates blank lines. To be precise, it inserts a newline by doing a **C-M**, and then backs up by doing a **C-B**. If dot is at the start of a line, this will leave dot sitting on the first character of a new blank line.

C-Q, C-X Q **quote**

Characters which are special to MicroEMACS can be inserted by using this command. The next character after the **C-Q** or **C-X Q** is stripped of any special meaning. It is simply inserted into the current buffer. Any argument specified on the **C-Q** or **C-X Q** command is used as the insert repeat count.

The **C-Q** form of the command is the easiest to use. However, some terminals demand that MicroEMACS perform XON/XOFF processing. If this is the case, the **C-Q** will be eaten by low level terminal support, and will not be usable as a command. The **C-X Q** form can always be used.

C-T **twiddle**

Exchange the two characters on either side of the dot. If the the dot is at the end of the line, twiddle the two characters before it. Does nothing if the dot is at the beginning of the line. This command is supposedly useful for correcting common letter transpositions while entering new text.

M-Tab **set-tab-size**

Set the tab size to the value of the argument, which must be a positive number greater than 1 (the default tab size is 8). This only affects the how MicroEMACS displays tabs on the screen; it does not affect how tabs are saved when a file is written to disk. To change how MicroEMACS handles tabs when saving a file, see the **set-save-tabs** command.

M-I **set-save-tabs**

By default, MicroEMACS preserves tabs when it writes a file to disk. If you pass a zero argument to this command, MicroEMACS will convert tabs to spaces when writing a file; the number of spaces is determined by the tab size (which you can set using **set-tab-size**). If you pass a non-zero argument to this command, MicroEMACS will revert back to the default behavior, which is to preserve tabs. With no argument, the setting is toggled.

M-C-T **trim-line**

Remove trailing spaces and tabs. With no argument, trim each line in the current region, or the current line when no region is marked; with an argument, trim that many lines.

C-X C-D **detab-line**

Convert tabs to spaces using the current tab size. With no argument, operate on the current region, or the current line when no region is marked; with an argument, operate on that many lines.

C-X C-E **entab-line**

Convert runs of spaces to tabs where they reach a tab stop. With no argument, operate on the current region, or the current line when no region is marked; with an argument, operate on that many lines.

M-+ **indent-region**

Shift each line in the current region by the argument. Positive arguments add indentation, and negative arguments remove indentation. Existing leading white space is replaced with the new indentation.

[unbound] **just-one-space**

If the dot is currently sitting over a tab or a space, all consecutive tabs and spaces to the left and right of the dot are deleted. Then a single space is inserted, and the dot is moved 1 character to the right.

7 Deleting and Killing

There are two general classes of commands that remove text from the buffer: delete commands and kill commands. Delete commands remove text from the buffer, and throw it away. Kill commands remove text from the buffer, but save the text in a special place called the *kill buffer* . Kill commands clear the kill buffer only if the previous command was not a kill command. Multiple kill commands executed sequentially append text to the kill buffer.

C-D **forw-del-char**

Delete characters to the right of dot. If an argument is specified that number of characters is deleted. If no argument is specified then 1 character is deleted. In addition, if an argument is specified, then the command kills the text instead of deleting it. It fails if there are not enough characters to delete between dot and the end of the buffer. On PCs, this function is also bound to `Del`.

Rubout,C-H,Backspace **back-del-char**

Delete characters to the left of dot. If an argument is specified then that number of characters is deleted. If no argument is specified then 1 character is deleted. In addition, if an argument is specified, the rubout command kills the text instead of deleting it. The command fails if there is not enough text between the start of the buffer and dot.

C-K **kill-line**

This is the basic killing command. If there is no argument it kills from dot to the end of the line, unless dot is at the end of line, when it kills the end of line. If a positive argument is specified, **C-K** kills that many lines, including the newlines. If an argument of 0 is specified, **C-K** kills from the start of the current line to dot. Finally, if a negative argument is specified, **C-K** kills backwards over abs(arg) newlines. This command fails if there aren't enough characters left in the buffer to be killed.

C-W **kill-region**

Kill all of the text enclosed in the region, and put it into the kill buffer. If an argument is specified, then delete the text instead of killing it; this is useful when memory fills up and MicroEMACS can't allocate space for the kill buffer. The value of the argument, if any, is ignored.

C-Y **yank**

Insert the text from the kill buffer into the current buffer at dot. If an argument is specified, it specifies the number of times the text is yanked. If no argument is specified, yank the text back once. Dot is advanced over the inserted text, as if the text had been typed in normally. Always succeeds.

M-W **copy-region**

Put all of the text enclosed in the region into the kill buffer, without deleting it from the current buffer. This is similar to **C-W** followed by **C-Y**, except that the buffer is not flagged as having been changed.

C-X C-O **del-blank-lines**

Delete blank lines around dot. If dot is sitting on a blank line, this command deletes all the blank lines above and below the current line. If it is sitting on a non blank line then it deletes all of the blank lines after the line. Any argument is ignored.

8 Rectangles

A *rectangle* is a column range of lines, cut on the *display-cell grid* : the grid a line renders on, where a tab counts as the cells up to the next tab stop and a control character as two. Set the anchor on one corner with **C-X r C-@** (**C-Space** sends the same key), put dot on the opposite corner, and run a command; the corners work in any order, and each command drops the anchor once it has used it. Or mark the corner with **C-Space** as in GNU Emacs: with no anchor set, the commands take the rectangle between mark and dot. While the anchor is set the rectangle between it and dot is highlighted, following dot, out past the end of a line that ends first, and while the **C-X r** prefix waits for its key the rectangle the command would take is highlighted the same way — the anchor's, or the mark's when no anchor is set. A character is part of the rectangle when its cells intersect the range, so one straddling an edge is taken whole; the commands that edit the buffer first expand a straddling tab into the spaces it renders as. Lines shorter than the rectangle read as blanks where text is copied. The rectangle clipboard is separate from the kill buffer. The prefix is **C-X r**, as in GNU Emacs, and the keys take the region verbs one prefix deeper — **C-w** kill, **M-W** copy, **C-y** yank — with **C-c** clear, **C-o** open, and **C-t** string from GNU's rectangle chords, and the anchor on **C-@**, the **set-mark** key, as GNU's rectangles run between point and mark.

C-X r C-@ **rectangle-anchor**

Drop the rectangle anchor at dot, remembering its line and column, or drop the anchor when one is already set. The anchor cannot be set past the last line. Editing that removes the anchored line drops the anchor; the next command reports "Rectangle anchor lost". **C-G** drops it too, taking the highlight with it.

C-X r C-c **clear-rectangle**

Blank the rectangle in place: its characters become spaces, one per cell they occupied, so nothing to the right moves. Dot stays at its column.

C-X r C-o **open-rectangle**

Insert a column of blanks the width of the rectangle, shifting the text to the right of it. A prefix argument gives the width of the opened column in place of the rectangle's own. Lines too short to reach the rectangle are padded out to its left edge first. Dot is left on the upper left corner of the opened column.

C-X r C-t **string-rectangle**

Prompt for a string, then replace the rectangle with it: each line's slice of the rectangle is removed and the string is inserted at its left edge. A line too short to reach the rectangle is padded out to its left edge first. Dot is left on the upper left corner of what was the rectangle, at the head of the string.

C-X r C-w **kill-rectangle**

Copy the rectangle as **copy-rectangle** does, then remove its characters from each line. Dot is left on the upper left corner of what was the rectangle. One **C-X U** undoes the whole command.

C-X r M-W **copy-rectangle**

Copy the rectangle between the corners into the rectangle clipboard, leaving the buffer alone. A line of the rectangle that ends before the right edge contributes blanks.

C-X r C-y **yank-rectangle**

Insert the rectangle clipboard at dot's column, one slice on dot's line and the lines below. A line that ends before the column is padded out with spaces, and the yank stops at the end of the buffer, reporting how many lines fit.

9 Words

MicroEMACS has commands that manipulate words. In full EMACS the characters that make up words can be changed by the user. In MicroEMACS, they are fixed, and include the upper and lower case letters, the dollar sign $ and the underline _. This set of characters is intended more for editing programs, but the word commands still work reasonably when editing text.

M-B **back-word**

The backward word command moves dot backward by words, stopping on the first character of the word. If an argument is specified, it is the number of words over which to move. The default argument is 1. On PC-DOS and FlexOS this function is also bound to `C-Left`.

M-C **cap-word**

This command moves forward over a word, converting all characters in the word to lower case except the first one, which is converted to upper case. If an argument is supplied, it must be positive, and it specifies the number of words over which to move.

M-D **forw-del-word**

The delete word command moves dot forward by words, and kills any characters over which it moves. If an argument is specified, it is the number of words over which to move. The default argument is 1.

M-F **forw-word**

The forward word command moves dot forward by words, stopping on the first non-word character. If an argument is specified, it is the number of words over which to move. The default argument is 1. On PC-DOS and FlexOS this function is also bound to `C-Right`.

M-L **lower-word**

This command moves forward over a word, converting it to lower case. If an argument is supplied, it must be positive, and it specifies the number of words over which to move.

M-C-H **back-del-word**

The backward delete word command moves backward by words, stopping on the first character of the word, and killing as it moves. If an argument is present, it specifies the number of words over which to move. The default argument is 1.

M-Rubout **back-del-word**

This command is the same as **M-C-H**. It exists only to preserve the symmetry between the word commands and the character commands (both backspace and rubout are backward delete character).

M-U **upper-word**

This command moves forward over a word, converting it to upper case. If an argument is supplied, it must be positive, and it specifies the number of words over which to move.

M-C-C **count-words**

Count the words, characters, and lines in the current region, and report the results on the echo line.

10 Paragraphs

MicroEMACS has commands that deal with paragraphs. A paragraph is a sequences of lines separated by one or more of the following:

- Blank lines.
- Lines beginning with a space or tab character.
- Lines beginning with the "@" character (SCRIBE, FinalWord, and Sprint command lines).
- Lines beginning with the "." character (NROFF and PROFF command lines).

There are commands for "filling" a paragraph, moving dot to the start or end of a paragraph, setting the fill column, and performing automatic word wrap.

[unbound] **set-fill-column**

This command sets the current fill column to its argument (remember that the argument is entered as `Control-U` and a decimal number preceding the command). If no argument is present, the column of the current location of the cursor (the dot) is used instead. The fill column is used by the **fill-paragraph** and **ins-self-with-wrap** commands. The default fill column is 70.

M-[**back-paragraph**

This command moves the dot to the beginning of the current paragraph. If the dot is not in a paragraph when this command is entered, it is moved to the beginning of the preceding paragraph. If an argument is provided, the dot is moved by that many paragraphs.

If you are using MicroEMACS with a serial terminal, you may have to type `ESCAPE` followed by two `[` characters to invoke this command. The reason is that ESCAPE-[is the prefix produced by function keys on VT-100 compatible terminals.

M-] **forw-paragraph**

This command moves the dot to the end of the current paragraph (actually to first separator line after the paragraph). If the dot is not in a paragraph when this command is entered, it is moved to the end of the following paragraph. If an argument is provided, the dot is moved by that many paragraphs.

M-J **fill-paragraph**

This command "fills" the current paragraph. It inserts or deletes spaces and line breaks between words as needed to cause the text of each line to be filled out to (but no farther than) the current fill column. The text is thus filled, but not right-justified. The dot is then placed at the end of the last line of the paragraph.

M-C-W **kill-paragraph**

This command deletes the current paragraph. If an argument is provided, that many paragraphs are deleted.

[unbound] **ins-self-with-wrap**

This command inserts the key that invoked it into the buffer at the current location of dot, and dot moves 1 character to the right. Then, if dot has gone past the current fill column, a line break is inserted in front of the first word in the line that extends past the fill column. Thus the current line is "filled", and a new line is created that contains the words from the current line that wouldn't fit within the fill column.

Normally, this command is not bound to any key. If you bind this command to the space key, the effect will be similar to the word-wrapping mode in MINCE.

11 Case Conversion

In addition to the word mode case conversion commands, MicroEMACS has commands to modify the case of large blocks of text. These commands should be used with caution because they cause major damage to (potentially) large areas of the buffer.

C-X C-L **lower-region**

The lowercase region command converts all of the characters between dot and mark into lower case.

C-X C-U **upper-region**

The uppercase region command converts all of the characters between dot and mark into upper case.

12 Searching and Spelling

Search commands move though the buffer, in either the forward or the reverse direction, looking for text that matches a search pattern. Search commands prompt for the search pattern in the echo line. The search pattern used by the last search command is remembered, and displayed in the prompt. If you want to use this pattern again, just hit carriage return at the prompt.

In search strings, all characters stand for themselves, and all searches are normally case insensitive. The case insensitivity may be defeated with the "fold-case" command, described below. The newline characters at the ends of the lines are considered to have hexadecimal value 0A, and can be matched by a linefeed (**Control-J**) in the search string.

A carriage return can be searched for by preceding it with `Control-Q` in the search string. On PC-DOS, CP/M and FlexOS this will not match the carriage return in the CR-LF character pair that normally terminates a line of text. It will only match a bare carriage return that has no following line feed.

MicroEMACS supports regular expression searches using a subset of POSIX regular expressions:

- `.` (dot) matches any character
- character classes (square brackets with optional ^ negation operator)
- groups (parentheses)
- alternatives (`|`)
- `^` (start of line) and `$` (end of line)
- `?` (zero or one occurrence)
- `*` (zero or more occurrences)
- `+` (one or more occurrences)

C-S RET **forw-search**

A **RET** pressed in incremental search before any search text has been typed drops into the plain, non-incremental search: prompt for a string, then search forward, from the current location, toward the end of the buffer. If found, dot is positioned after the matched text. If the text is not found, dot does not move.

The **C-S** form of the command is not usable if the terminal being used required XON/XOFF support. In fact, if you use this format of the command on such a terminal, it will hang until you type **C-Q**. **M-S**, the `F1` key on the Z-29 keyboard, starts the incremental search like **C-S**.

C-R RET **back-search**

A **RET** pressed in reverse incremental search before any search text has been typed drops into the plain, non-incremental reverse search: prompt for a string, then search in reverse, from the current location, toward the front of the buffer. If found, dot is positioned at the first character of the matched text. If the text is not found, dot does not move.

M-C-S **forw-regexp-search**

Similar to **forw-search**, except that the search string is a regular expression, and searches cannot cross line boundaries.

M-C-R **back-regexp-search**

Similar to **back-search**, except that the search string is a regular expression, and searches cannot cross line boundaries.

M-C-F **fold-case**

Enable or disable case folding in searches, depending on the argument to the command. If the argument is zero, case folding is disabled, and searches will be sensitive to case. If the argument is non-zero, case folding is enabled, and searches will NOT be sensitive to case. With no argument, the setting is toggled.

M-P **search-paren**

Search for a match of the character at the dot. If the character is a parenthesis or bracketing character, move the dot to the matching parenthesis or bracket, taking into account nesting and C-style comments. A parenthesis or bracketing character is one of the following: (){}[]<>

Pad-1,Shift-5 **search-again**

Repeat the last forw-search command (**C-S, M-S**) or prev-search command (**C-R, M-R**), without prompting for a string. This command is bound to two keys on the numeric keypad of the Z-29 terminal: the `1` key, and the shifted `5` key. On PCs, this function is also bound to `F9`.

C-S **forw-i-search**

Enters incremental search mode, with the initial search direction being forward. In incremental search mode the following keys are recognized.

- **C-N** finds the next occurence of the string (if it is first thing typed, reuse the previous string).
- **C-P** finds the previous occurence of the string (if it is the first thing typed, reuse the previous string).
- **C-S** (or **C-F**) switches the search direction to forward, and finds the next occurrence of the string.
- **C-R** (or **C-B**) switches the search direction to reverse, and finds the next occurrence of the string.
- **C-Q** (or **C-^**) quotes the next character (allows searching for C-N, etc.).
- **C-W** appends the rest of the word after dot to the search pattern.
- **RET** exits like **ESC**; pressed before any text has been typed, it drops into the plain **forw-search** or **back-search** instead.
- **ESC** exits from incremental search mode.
- **C-G** restores dot to original location before incremental search mode was entered, then exits from incremental search mode.
- **DEL** undoes the effect of the last character typed in the search string.
- **C-U**, **C-X**, **C-J**, and all non-Control characters accumulate into search string.
- All other control characters exit from incremental search mode and are interpreted as normal commands.

C-R **back-i-search**

Enters incremental search mode, with the initial search direction being reverse. Otherwise identical to **C-S**.

M-Q,M-% **query-replace**\index{M-%}

Search and replace with query. This command prompts for a search string and a replace string, then searches forward for the search string. After each occurrence of the search string is found, the dot is placed after the string, and the user is prompted for action. Enter one of the following characters:

- **space** or **,** (comma) causes the string to be replaced, and the next occurrence is searched.
- **.** (period) causes the string to replaced, and quits the search.
- **n** causes the string to be skipped without being replaced, and the next occurrence is searched.
- **!** causes all subsequent occurrences of the string to be replaced without prompting.
- **Control-G** quits the search without any further replacements.

Normally this command adjusts the capitalization of the new string to match the old string when it performs a replacement. You can defeat this "feature" if you prefix this command with an argument (the argument value is ignored).

C-X % **replace-region**\index{C-X %}

Prompt for a search string and a replacement string, then query-replace only within the current region. At each match, answer **y** or **space** to replace it, **n** to skip it, **!** to replace all remaining matches, or **C-G** to stop.

M-R **replace-string**

Prompt for a search string and a replacement string, then search forward for all occurrences of the search string, replacing each one with the replacement string. Do not prompt the user for confirmation at each replacement, as in the **query-replace** command.

Normally this command adjusts the capitalization of the new string to match the old string when it performs a replacement. You can defeat this "feature" if you prefix this command with an argument (the argument value is ignored).

M-? **reg-query-replace**

Similar to **query-replace**, except that the search string is a regular expression, and the replacement string can contain the following special characters:

- `&` stands for the entire matched string.
- `\n`, where `n` is a digit in the range 0-9, stands for the nth matched group (where groups are delineated by parentheses in the regular expression pattern).
- `\` followed by either `\` or `&` stands for that character itself, without the leading `\`.

M-/ **rep-replace**

Similar to **reg-query-replace**, except that the user is prompted to confirm each replacement, as in **query-replace**.

C-X I **spell-region**

This command uses `ispell` to spell-check the current region (the text between the mark and the dot). At each misspelled word, MicroEMACS prompts for an action:

- **q** or **C-G** aborts the spell checking.
- **Space** ignores the misspelled word
- **a** ignores the misspelled word and adds it to ispell's list of words to ignore in the future.
- **0** to **9** replaces the misspelled word with one of up to ten suggestions; the suggestions are shown in the prompt on the echo line.
- **r** prompts for a string to replace the word.

M-# **spell-word**

Similar to **spell-region**, except that it checks only the word under the cursor.

13 Files

C-X C-F **find-file**

Prompt for a file name, then select that file for editing. If a buffer is already associated with the file, switch to that buffer; otherwise create a buffer, read the file into it, and switch to the buffer.

M-@ **set-file-name**

This command prompts in the echo line for a file name, which becomes the new associated file name for the current buffer.

C-X C-R **file-read**

This command prompts in the echo line for a file name, then it deletes all of the text in the current buffer and reads in the file. The associated file name is set to the name of the file just read. The number of lines read is displayed in the echo line.

C-X C-I **file-insert**

This command prompts in the echo line for a file name, then reads in the file, inserting its contents at the dot in the current buffer. The number of lines read is displayed in the echo line. The file name associated with the current buffer is not changed.

C-X C-S, M-T **file-save**

This command writes the contents of the current buffer to its associated file. The "changed" flag for the current buffer is reset. It is an error to use this command in a buffer which lacks an associated file name. This command is a no-operation if the buffer has not been changed since the last write. In a hexl buffer it writes the file as raw bytes parsed out of the dump, as the **Hexl Editing** section describes.

If you used the **-b** option when you invoked MicroEMACS, and this is first time a **file-save** command has been performed on the file, MicroEMACS will create a backup of the file. See the section **Starting** for more information on backups.

The **C-X C-S** form of the command is not usable if the terminal being used required XON/XOFF support. In fact, if you use this format of the command on such a terminal, it will hang until you type **C-Q**.

The **M-T** form of this command is easily entered on the Z-29 keyboard by pressing the `F2` key. On PCs, this function is also bound to `F2`.

C-X S **file-save-all**

Save the changed buffers that are editing files, as GNU Emacs's **C-x s** does. Each changed buffer with a file name is offered in turn — the buffer list runs newest first — with a "Save file (name) [y/n]?" prompt: answer **y** to write it out, **n** to leave it changed, and **C-G** to stop the tour with the buffers left as answered so far. Buffers with no file name are passed over. A prefix argument saves every changed buffer without asking. The current buffer is put back when the tour ends. A hexl buffer on the tour saves its bytes, as **C-X C-S** does in one.

C-X C-V **file-visit**

This command selects a file for editing. It prompts for a file name in the echo line. It then looks through all of the buffers for a buffer whose associated file name is the same as the file being selected. If a buffer is found, it just switches to that buffer. Otherwise it creates a new buffer, (fabricating a name from the last part of the new file name), reads the file into it, and switches to the buffer.

If the desired new buffer name is not unique (perhaps you tried to visit a file in some other directory with the same name as a file already read in) the command will prompt for a new buffer name. You can either supply a buffer name, or just type newline to overwrite the old buffer.

On PCs, this function is also bound to `F3`.

C-X C-W **file-write**

This command prompts in the echo line for a file name, then it writes the contents of the current buffer to that file. The "changed" flag for the current buffer is reset, and the supplied file name becomes the associated file name for the current buffer.

[unbound] **revert-buffer**

Reread the file associated with the current buffer, replacing the buffer contents and clearing the undo history. Unless the command is forced with an argument, MicroEMACS asks for confirmation when the buffer has changed.

M-$ **set-newline**

Toggle whether the current buffer uses **LF** or **CRLF** line endings. A numeric argument sets the ending rather than toggling it — nonzero for **CRLF**, zero for **LF**. The current setting is shown by **C-X =** and used when the buffer is saved.

C-X @ **pipe-command**

Prompt for a shell command, run it with standard output and standard error redirected to a temporary file, and display the output in the `*pipe*` buffer.

C-X # **filter-buffer**

Prompt for an external filter command. Write the current buffer to a temporary file, run the filter on that file, and display its output in the `*filter*` buffer.

C-X $ **change-dir**

Change MicroEMACS's current working directory. Supplying an empty path displays the current directory.

C-X D **dired**

Prompt for a directory, run a long directory listing, and display it in the `*dired*` buffer. When `lisp/dired.l` is loaded, this key instead opens the shell-free Lisp Dired implementation; the native command remains available as **M-X dired**.

14 Hexl Editing

C-X C-H **hexl-find-file**

Prompt for a file name, then read the file as a hex dump: one line per sixteen bytes, an eight digit offset, the bytes in hex in two groups of eight, and the printable characters beside them, with `.` for a byte that is not printable. The buffer takes the name of the file, the mode line shows `(hexl)`, and a file already dumped is revisited, as **C-X C-F** revisits its buffer. The ASCII area is display only; saving reads the hex. Invoking MicroEMACS as `me -H file` opens the file this way on startup.

00000000: 48 65 6c 6c 6f 2c 20 57 6f 72 6c 64 21 0a 00 01 Hello, World!. . .

The blank slots that pad the last line hold no bytes, so a file always dumps and saves at the same length.

Bytes are edited by typing over them, in either column. In the hex area two hex digits write the byte, and the first stays on display until the second arrives; in the ASCII area a character writes its own value. Both columns follow each other, and the dot steps to the next byte once a byte is written; TAB steps to the next byte without writing. **C-X C-S** saves the dump by parsing the hex back out of the lines and writing the file as raw bytes, and it is that save which the **quit** and **file-save-all** commands carry out on a hexl buffer; **C-X C-W** is refused, as it would write the dump instead of the bytes. Commands that are not hexl aware, as backspace and the kills, can damage a line; the save checks every line before the file is touched and refuses a damaged one by its number, and **undo** puts the line back.

The `hexl-goto-address` extended command prompts for a byte's offset in the file, as decimal or as hex with a `0x` prefix, and moves the dot onto it; a prefix argument is taken as the address.

The layout of the dump changes at run time. The `hexl-group-size` extended command sets how many bytes a group holds, `1`, `2`, `4`, `8` or `16`; a prefix argument is taken as the size. The dump opens in the GNU layout, the bytes spaced as pairs in two groups of eight; a group size packs each group into a single hex number, as `xxd`, Vim's hex dumper, packs its `-g` groups — a byte pair, a word, a doubleword, a quadword, an octaword — with the group's padding split to either side of it, so a group reads as the value it holds. The GNU layout is left once a group size is set; the `hexl-packed` extended command returns to it, and dumping the file anew opens in it again. The hex area gives each byte three columns whichever group size is set, so the line width and the ASCII column never move. The `hexl-endian` extended command switches the byte order within a group: big-endian, where the bytes read in file order, is the default, and little-endian shows each group reversed, as a two byte or four byte integer reads on the machine. A prefix argument of zero asks for big-endian, any other value for little-endian, and without one the order is

toggled. The `hexl-packed` extended command switches the groups between that packed form and the GNU layout: a prefix argument of zero asks for the GNU layout, any other value for the packed groups, and without one the layout toggles. Each command re-lays the dump out in place, keeping every byte, so edits survive the change; the ASCII area keeps the file order in any layout. A group cut short by the end of the file is never reversed, so its bytes read in file order either way. The settings hold per buffer and become the defaults for files dumped later in the session. A relaid dump cannot be undone — the change is one of layout, not of bytes, and the undo records speak the old layout — so the relaying clears the undo history of the buffer.

```
00000000:   6c6c6548    57202c6f    646c726f    01000a21  Hello, World!...
```

A file of more than 8 MB is refused with the limit in the message, as a dump of it would hold more than half a million lines.

15 Keyboard Macros

Keyboard macros simplify a large class of repetitious editing tasks. The basic idea is simple. A set of keystrokes can be collected into a group, and then the group may be replayed any number of times.

There is only one keyboard macro. However, you can effectively have more than one macro by giving the current keyboard macro a name. Otherwise, when you define a new keyboard macro, the old one is erased.

You can also save macros for use in future editing sessions, with the ins-macro function.

C-X (**start-macro**

This command starts the collection of a keyboard macro. All keystrokes up to the next **C-X)** will be gathered up, and may be replayed by the execute keyboard macro command.

C-X) **end-macro**

This command stops the collection of a keyboard macro.

C-X E **execute-macro**

The execute keyboard macro command replays the current keyboard macro. If an argument is present, it specifies the number of times the macro should be executed. If no argument is present and the mark is set, the macro runs once for each line in the region and dot moves to the beginning of the region. Otherwise, it runs the macro once. Execution of the macro stops if an error occurs (such as a failed search).

[**unbound**] **name-macro**

This command saves the current keyboard macro and gives it a name. MicroEMACS prompts you for the name to assign to the macro. The macro name can then be used as an extended command (using `ESC X`), or can be bound to a key using **bind-to-key**, just as a normal command.

[**unbound**] **ins-macro**

This commands inserts the contents of a macro into the current buffer in a format that can be used later in a profile. MicroEMACS prompts for the name of the macro. If you don't enter a name, the current macro is used.

16 Buffer Management

Previous sections have made references to the text in "the buffer", which implied that there is only one buffer. This is not true; MicroEMACS allows any number of buffers, memory space permitting.

Each buffer has its own buffer name (a 16 character string), and optional associated file name, and a block of text. A value of dot and mark is also associated with any buffer that is not currently being displayed. This remembered value of dot and mark makes a buffer come back in approximately the same state as it was when it was hidden.

Also associated with each buffer is a changed flag. This flag is set when the text in the buffer is modified, and reset when the text in the buffer is written out to its associated file. MicroEMACS will always ask for confirmation before executing a command that would cause changed text to be lost.

C-X C-B **buffer-menu**

Open a full-screen buffer menu. Move to an entry with **n**, **p**, the arrow keys, or **space** and **backspace**. The menu keys are:

- **1** or **Return** selects the buffer in the current window.
- **2** selects the buffer in a split window below the original window.
- **s** saves the buffer on the selected line.
- **v** toggles the buffer's read-only flag.
- **k** or **d** kills the buffer on the selected line.
- **q**, **Q**, **x**, or **X** closes the menu and returns to the original buffer.

[**unbound**] **display-buffers**

Create a pop-up window containing the name, size, associated file name, and changed flag of every buffer. Unlike **buffer-menu**, this display is not interactive. On PCs, this function is also bound to `F6`.

C-X B **use-buffer**

This command prompts for a buffer name, and then switches the buffer being displayed in the current window to that buffer. The buffer will be created if it does not exist.

If you do not enter a buffer name, this command will use the name of the last buffer that you switched from with **C-X B**. Thus, you can use **C-X B** repeatedly to switch between two buffers without entering their names each time.

[**unbound**] **forw-buffer**

This command switches the buffer being displayed to the next buffer in the buffer list. If the end of the buffer list is reached, switch to the first buffer in the list. The list of buffers can be displayed with **C-X C-B**. When this command is bound to a key, it is useful for quickly flipping among the files being edited. On PCs this command is bound to `F8`.

[**unbound**] **back-buffer**

This command is similar to **forw-buffer**, except that it switches the buffer being displayed to the previous buffer in the buffer list.

C-X K **kill-buffer**

This command prompts for a buffer name, and then destroys the buffer with that name. It will ask for permission to destroy the buffer if the text has been changed since it was written to the associated file. You cannot delete a buffer that is being displayed.

C-X C-Q **toggle-readonly**

This command toggles the read-only flag on the current buffer: if the buffer is currently read-only, it is made read-write; otherwise it is made read-only. A numeric argument sets the flag rather than toggling it — nonzero makes the buffer read-only, zero makes it read-write. This can be useful to counteract the effect of starting MicroEMACS with the `-r` option.

M-~ **umark-buffer**

Clear the changed flag for the current buffer. MicroEMACS will no longer consider the buffer modified, so use this command carefully.

17 Window Management

MicroEMACS lets you have multiple windows on the screen. Each window has its own mode line, its own value of dot and mark, and its own associated buffer. If you have a buffer displayed in more that one window and you edit a line, it is updated in all windows.

A window is as only wide as can fit on your terminal. If a text line is too long to be displayed in a window, the last column is displayed as a `$` character. However, MicroEMACS will let you view long lines by automatically scrolling the window left or right if you move the cursor past the left or right edges of the window. When MicroEMACS performs a horizontal scroll, it will attempt to reframe the window so that the cursor is centered horizontally.

C-L **refresh**

This command clears the screen, and completely redisplays all of the text is all of the windows. It is useful if a line error has garbaged your screen.

If you give **C-L** an argument, it will attempt to split the screen vertically into that number of side-by-side pages. This has the effect of giving you a screen that has more rows but fewer columns than normal. To restore the screen to normal, give an argument of 1. Currently this feature only works with the ncurses display code; see **Building with GCC**.

C-X 2 **split-window**

This is the basic window making command. The current window is split in two. Each window is displaying the same buffer, and has the same value of dot and mark. The window must be at least three lines high, otherwise there isn't enough room for two text lines and the new mode line. Any argument is ignored.

After the you create a window, you usually switch it to a private buffer using **C-X B** or (perhaps more often) **C-X C-V**. Note that because both windows are displaying the same buffer, reading a file in one window with **C-X C-R** probably does not do what you want.

C-X 1 **only-window**

This is the basic window destroying command. All windows but the current window are removed from the screen. The current window grows to fill the vacated screen space. Any argument is ignored. On PCs, this command is also bound to `F10`.

C-X 0 **zap-window**

Remove the current window and give its space to an adjacent window. If there is only one window, report an error instead. The current buffer is not killed.

C-X N, C-X O **forw-window**

Move to the next window down the screen. If the current window is the bottom window, move to the top window. Ignores any argument.

C-X P **back-window**

Move to the previous window up the screen. If the current window is the top window, move to the bottom window. Ignores any argument.

C-X Z **enlarge-window**

The current window is enlarged, if possible. Any argument is used as a "number of lines by which to grow". The default argument is 1. Screen space is stolen from the window immediately below the current window. If the current window is the bottom window on the screen then space is stolen from the window immediately above the current window. You cannot steal all of the lines away from a window.

C-X C-N **down-window**

Scroll the current window down. Any argument is used as a "number of lines by which to scroll" count. The default argument is 1. On PCs, this command is also bound to `C-PgDn`.

C-X C-P **up-window**

Scroll the current window up. Any argument is used as a "number of lines by which to scroll" count. The default argument is 1. On PCs, this command is also bound to `C-PgUp`.

C-X C-Z **shrink-window**

The current window is shrunk, if possible. Any argument is used as a "number of lines by which to shrink". The default argument is 1. Screen space is given to the window immediately below the current window. If the current window is the bottom window on the screen, the space is given to the window immediately above the current window. You cannot shrink a window out of existence.

M-C-V **scroll-next-down**

Scroll the next window forward one page, or by the argument number of lines, while leaving dot in the current window.

M-C-Z **scroll-next-up**

Scroll the next window backward one page, or by the argument number of lines, while leaving dot in the current window.

M-! **reposition-window**

This command is used to control the line of a window upon which dot is displayed. If its argument is positive, then that number is taken to be the origin 1 line number of the current window upon which the line containing dot should be placed. If its argument is 0, then dot is moved to the center of the window. If the number is less then zero then it is taken to be the negation of the origin 1 line number of the current window starting at the bottom upon which the line containing dot should be placed. If no argument is supplied, a default argument of 1 is used; This lets **M-!** function as a "move dot to the top of the window" command, which is very useful.

C-X + **balance-windows**

This command adjusts the windows so that they all have approximately the same height. This is useful after several **split-window** commands have created some windows that are too small.

18 Messages

MicroEMACS has a message system that allows commands to display error or status messages that are too long to fit on the echo line. At present, only the **display-version** command uses the message system, but other commands may use it in the future.

[**unbound**] **display-message**

Display the message lines one at a time on the echo line, then enter a special message display mode, in which certain keys have the following meanings:

- **C-N, Space, Return**: go forward one line, and quit if the end of the message has been reached.
- **C-P, C-H, Backspace**: go backward one line.
- **C-G**: quit, leave the message unchanged.
- **C-C**: quit, erase the portion of the message already read, but leave the remainder of the message unchanged.

C-X V **display-version**

Copy the version strings into the message buffer, then execute a **display-message** command (see above). Use this command to find out which version of MicroEMACS you are running.

19 Key Binding Commands

There are several commands in MicroEMACS that can be used to examine or change key bindings. These commands are not normally bound to a particular key. As with any unbound commands (also called "extended" commands), they can be used by entering **M-X** (`ESC X`), then typing the command name in response to the prompt `:` on the echo line.

[unbound] **display-bindings**

This command creates a pop-up window that contains a table of the current keyboard bindings. Each entry in the table contains the key name, and the name of the command that the key invokes. You can save this table to a file by using **C-X N** to switch to the window displaying the table, then using **C-X C-W** to write the table to a file.

[unbound] **bind-to-key**

This command prompts the user with `Function:` for a command name, then prompts with `Key:` for a key (or key combination). The command is then bound to that key, so that all subsequent uses of that key cause the associated command to be executed. The command name can be any of those listed by the **display-bindings** command, or any of the commands described in this section.

[unbound] **help**

This command waits for the user to enter a key, then displays on the echo line the command name that is bound to that key. On PCs, this function is bound to `F1`.

20 Tags

MicroEMACS can use the source-code tagging programs `cscope` and `ctags` to make it easier to find functions and variables in C programs. The `find-` functions prompt for a name to find, but they also attempt to extract a name from the current buffer and location, to be used if you do not enter a name and simply hit `Enter`.

M-, **find-cscope**

This command prompts for an identifier, then uses `cscope` to find the first occurence of the identifier. Usually, the first occurrence is the one that defines the identifier. The command then visits the corresponding file and places the dot at the line containing the identifier. On PCs, this function is also bound to `F11`.

M-G **find-grep**

This command prompts for a string, then uses `cscope`'s "grep" function to find the first occurence of the string. The command then visits the corresponding file and places the dot at the line containing the string.

[unbound] **next-cscope**

This command uses `cscope` finds the next occurrence of an identifier that was found in a previous `find-cscope` or `find-grep` command. On PCs, this function is bound to `F12`.

M-. **find-tag**

This command prompts for an identifier, then reads the `TAGS` file (generated by `ctags`) to find an occurrence of the identifier. If no argument is present, the command finds the first occurrence; if an argument is present, the command finds the next occurence. Use `C-u M-.` to find the next match and `M-- M-.` to find the previous match. The command then visits the corresponding file and places the dot at the line containing the identifier.

M-C-. **find-tag-regexp**

This command prompts for a regular expression, then searches tag names in the `TAGS` file. Its argument behavior matches `find-tag`: no argument starts from the first tag, `C-u M-C-.` finds the next match, and `M-- M-C-.` finds the previous match. Regular-expression and identifier tag searches keep separate patterns and match positions.

C-X 4 . **find-tag-other-window**

This command behaves like `find-tag`, but selects the next window; if there is only one window, it splits the window only after a matching tag is found. `C-X 4 M-C-.` runs `find-tag-regexp-other-window` and behaves like `find-tag-regexp`.

M-* **pop-tag-mark**

This command returns to the buffer and location from which the most recent `find-tag`, `find-cscope`, or `find-grep` navigation started. Repeating the command retraces earlier origins; MicroEMACS keeps the sixteen most recent origins. If a line has become shorter since the jump, the saved offset is clamped to the end of that line.

21 Error navigation

MicroEMACS is able to parse a buffer containing compiler error messages, standard `filename:line` locations, or cscope-style matches, and load the source files mentioned in those messages. Typically, you would save compiler error messages to a file when running `make`:

```
make >&errs
```

Then load the resulting `errs` file into MicroEMACS and use either of the following equivalent commands to process the messages contained within:

M-C-E **goto-error**

C-X ‘ **next-error**

These commands look for the next location in the current buffer. If one is found, then a new window is opened if there is currently only one window; otherwise the next window below the current one is used. MicroEMACS reads into that window the file indicated in the location, positions the cursor at the line and column given in the location, and displays the remainder of the message in the echo line. To find the next error location, switch back to the error buffer (using the `back-window` command (**C-X P**) or the `forw-window` command (**C-X N**)), and issue either command again.

22 Undo

MicroEMACS supports an undo/redo facility. It saves information about the last 100 commands that modified a given buffer, so that these commands can be undone. Each buffer has its own set of undo records. MicroEMACS treats consecutive typing of normal (non-command) keys as a single operation; this makes it less tedious to undo large amounts of typing.

The most recent undo operation(s) can be reversed with the `redo` command. By using `undo` and `redo` in succession, you can go backwards and forwards in time through the recent history of a buffer. But if you modify the buffer in any way other than through `undo` or `redo`, the ability to `redo` will be lost until another `undo` is performed. This prevents conflicting changes from being made to a buffer.

C-X U **undo**

This command undoes the most recent operation that modified the current buffer. On PCs, this function is also bound to `F5`.

[**unbound**] **redo**

This command undoes the most recent undo. On PCs, this function is bound to `F7`.

23 Profiles

A profile is a file that contain a sequence of characters that MicroEMACS can read as if the file were a keyboard. Thus a profile is similar to a macro, but because it is a file it doesn't go away when you leave MicroEMACS. You can think of a profile as an editor command file.

When MicroEMACS starts, it reads and executes a startup profile, after it reads the file(s) you specify on the command line (if any). You might use the startup profile to set up some favorite key bindings, or define a macro, or read in more files. Before profile commands are executed, MicroEMACS loads its Lisp startup file, if one is present. See the **Lisp Extensions** section for details.

You can use the **-p profile** option when you invoke MicroEMACS to specify the name of the startup profile. If you don't use this option, MicroEMACS will use a default profile name. Use the **-q** option to skip the default profile and the default Lisp startup file; explicitly selected **-p** and **-l** files are still loaded. The name of the default profile is `.mepro` on UNIX-like systems and `me.pro` on Windows. MicroEMACS will look for this profile in the current directory. If the profile is not found, MicroEMACS will look in a second directory, as follows:

- On UNIX and PC-DOS, the directory indicated by the environment string **HOME**, if defined.
- On FlexOS, the **home:** directory or device.
- On CP/M-68K, user area 0 of the current drive.
- On VMS, the **sys$login** directory.

A profile consists of a series of tokens separated by white space or newlines. A token can be a literal string, key name, or command name.

1. **Literal string:** a series of characters surrounded by double quotes ("). The characters within the quotes are interpreted by MicroEMACS exactly as if you had typed them on the keyboard. Certain control characters may be placed in the string using the following escape sequences:

\n	linefeed
\t	tab
\b	backspace
\r	carriage return
\f	form feed
\\	backslash
\"	double quote

   Other control characters can be placed in the string by preceding them with the Control-Q (quote) character.

   A quoted string must always follow a command that normally prompt the user to enter a response string (for example, the **M-R** replace-string command). The last character in the response string must be a carriage return (written as \r).

2. **Key name:** the name of a MicroEMACS key surrounded by brackets []. Key names use the conventions established in this manual. Examples:

[C-F]	means "Control-F"
[M-C-V]	means "ESC Control-V"
[M-L]	means "ESC L"
[C-X E]	means "Control-X E"
[C-X C-S]	means "Control-X Control-S"

   You can use the **display-bindings** extended command to get a partial list of key names, or see the **Wall Chart** section below.

   MicroEMACS converts the key name to the corresponding internal key code. Only one key name is allowed within a single pair of brackets.

3. **Commands name:** simply the name of any MicroEMACS command, whether it is bound to a key or not. MicroEMACS prefixes the command name with **ESC X**, and follows it with a **Return**, before interpreting the command. This simulates the keystrokes that you would enter at the keyboard to invoke an extended command.

4. **Decimal number:** a series of digits, optionally preceded by a minus sign ("-"). It is equivalent to typing Control-U, followed by the number, on the keyboard. Placing a number before a command is a convenient method of supplying a numeric argument to the command.

As an example, consider the following line from a profile:

```
bind-to-key "help\r" [m-h]
```

This is equivalent the following key sequence typed at the keyboard:

```
ESC X bind-to-key RETURN help RETURN ESC H
```

Note especially the '\r' character in the quoted string: this must be present because MicroEMACS always expects you to type **Return** in response to the `Function:` prompt of the **bind-to-key** command. This is true for all commands that prompt on the echo line for a reply.

Here is a profile that moves the cursor down by 10 lines, and which demonstrates the use of a numeric argument to a command:

```
10 forw-line
```

Here is a profile that changes all occurrences of the string "plugh" to "xyzzy" in the current file, then saves the changes and quits.

```
replace-string\index{replace-string} "plugh\r" "xyzzy\r" file-save quit
```

You can automate this kind of global change with the **-p** option when you invoke MicroEMACS. If you name the above profile `junk.pro`, you can perform a global change on a file, without entering any MicroEMACS commands, by invoking MicroEMACS with the following:

```
me -p junk.pro filename
```

The historical `logline.pro` file at the root of the source tree contains only the macro-definition line shown below. Loading it with `me -p logline.pro filename` defines the current macro and changes one line; press **C-X E** to replay it on the next line.

The source tree also contains a complete profile example in `examples/logline.pro`. It defines a macro that changes a line to `TAB * text:` format, gives the macro the name `format-log-line`, and binds it to `M-L`:

```
[C-X (] [C-A] [Tab] "* " [C-E] ":" [C-N] [C-A] [C-X )]
name-macro "format-log-line\r"
bind-to-key "format-log-line\r" [M-L]
2 format-log-line
file-save
quit
```

The first line both defines and executes the macro once: profile input is interpreted as keystrokes, so the definition changes the current line and leaves dot on the next line. Therefore `2 format-log-line` executes the macro for the two remaining sample lines, for three changed lines in total.

With the three-line file `examples/logline-input.txt`, run:

```
cp examples/logline-input.txt /tmp/logline.txt
me -p examples/logline.pro /tmp/logline.txt
```

The saved file contains:

```
TAB * first:
TAB * second:
TAB * third:
```

You can tell MicroEMACS to read a profile at any time, with the following command.

C-X F **read-profile**

This command prompts you for the name of a profile. MicroEMACS then reads its subsequent commands from the specified profile, until the end of the file is encountered. While a profile is being processed, command errors are reported, but otherwise no screen activity takes place (unless the "echo" command is used in the profile). You cannot nest profiles by putting a [C-X F] or read-profile command in a profile. Execution of a profile does *not* stop if an error occurs (such as a failed search).

When Lisp support is enabled, files ending in `.l`, `.lisp`, or `.mel` are evaluated as Lisp source instead of profiles. Use **M-X read-lisp** to evaluate a Lisp file explicitly.

You can display messages on the echo line during profile processing (or at any other time) with the following command.

[**unbound**] **echo**

This command prompts you to enter a line of text. MicroEMACS then displays the text on the echo line. This command might be useful for displaying progress during a time-consuming profile. Here is an example of the use of the echo command in a profile:

```
echo "Changing all CDOS references...\r"
replace-string "CDOS\r" "FlexOS\r"
```

Note the required carriage return (\r) at the end of the string.

24 Lisp Extensions

MicroEMACS includes a small built-in Lisp evaluator based on Tiny Lisp. Start it with **M-X eval-lisp**; no key is bound to the command by default. The evaluator accepts one expression at a time. Its environment persists between expressions, so functions, macros and variables can be defined and used in later expressions. It supports numbers, strings, symbols, lists, lexical scopes, optional function parameters, functions, macros, tail recursion, arithmetic, comparisons, list operations, function references, `let`, `while`, `dotimes`, `unless`, and output primitives:

```
(+ 1 2)
(defun plus1 (x) (+ x 1))
(plus1 2)
(defmacro inc (x) (list '+ x 1))
(inc 2)
```

These forms also provide access to MicroEMACS state and the current buffer:

```
(define-key "C-x C-b" "buffer-menu")
(define-mode-key "C-m" "my-command")
(set-variable "tab-width" 8)
(message "Done: %s" "ok")
(load "library.l")
(require "smart-indent")
(set-lisp-debug 2)
(debug-message 1 "Done: %s" "ok")
(set-syntax "sql")
(set-modeline-color 15 22 0 2)
(set-selection-color 15 24 7 4)
(execute-command "file-save")
(me-find-file "file.c")
(get-char)
(get-line)
(get-line -1)
(get-region)
(count-char "{}" "{")
(insert-string "text")
(insert-char "X")
(save-buffer)
(kill-emacs)
(goto-bol)
(goto-eol)
(forward-line 2)
(eobp)
(define-lisp-command "format-log-line" #'format-log-line)
(switch-to-buffer "*dired*")
```

```
(buffer-file-name)
(string-suffix-p "directory/" "/")
(string-trim-suffix "directory/" "/")
(join-path "directory" "file")
(resolve-path "directory/link")
(read-directory-name "Dired" ".")
(read-file-name "Find file" "file.c")
(set-buffer-readonly t)
(create-mode "Dired")
(dired-list "directory")
(dired-entry-name)
```

Builds with AI enabled add these forms:

```
(getenv "API_KEY")
(ai-complete "current line")
```

`define-key` binds a key to a MicroEMACS command. Key names support the **C-X**, **C-** and **M-** prefixes followed by one ASCII character. `define-mode-key` binds a key in the current buffer's mode map; it fails when the buffer has no mode. `create-mode` creates or replaces that mode map. `set-variable` sets **tab-width** (or **tab-size**), **fill-column**, or **mouse-wheel-step** to a positive integer; a quoted string is also accepted for the value. `message` formats text with `%s`, `%S`, `%d`, and `%%`, displays it on the echo line, and returns the formatted string. `load` evaluates a Lisp file in the persistent global environment and returns `t`. A relative name without a directory separator is searched for in the current directory, `~/.mel.d/`, `~/.config/me/lisp/`, `/usr/local/share/me/lisp/`, and `/usr/share/me/lisp/`. `require` accepts a feature name as a string or symbol, loads it once, and returns `t` for every successful or already-completed request. It tries `feature.l`, `feature.lisp`, and `feature.mel` in the same directories. The optional GNU Emacs-style arguments are a filename and `NOERROR`; when `NOERROR` is non-nil, a missing module returns `nil` instead of raising a Lisp error:

```
(require 'smart-indent)
(require "smart-indent")
(require "smart-indent" "path/smart-indent.l")
(require "smart-indent" nil t)
```

`set-lisp-debug` sets the Lisp debug level; level 0 disables debug output. `debug-message` takes a positive message level, a format string, and optional arguments. It uses the same conversions as `message`, displays `[Lisp debug: ...]` when its level is less than or equal to the current debug level, and returns `t`. A final `t` result is quiet unless the debug level is nonzero. `set-syntax` selects a syntax definition for the current buffer and is available when color is enabled. `execute-command`, also available as `editor-cmd`, runs a MicroEMACS command by name with an optional numeric argument; use direct buffer forms when a command would otherwise prompt for text. `set-modeline-color` customizes the mode line when color is enabled. The first two arguments are the foreground and background 256-color indexes; the last two are the fallback colors for 8/16-color terminals and the Windows console. All color indexes must be from 0 through 255. `set-selection-color` sets the mouse selection highlight the same way in builds with the mouse compiled in; the default is 15 24 7 4, bright white on a deep blue background instead of reverse video.

`switch-to-buffer` finds or creates a named buffer and makes it current. `buffer-file-name` returns the current buffer's file name as a string. `string-suffix-p` tests a string suffix, `string-trim-suffix` removes one, and `join-path` joins a directory and file name with the correct separator. `resolve-path` returns the canonical path for an existing file or directory, or nil when the path cannot be resolved. `read-directory-name` prompts for a directory with filename completion and uses its optional second argument as the default; `read-file-name` does the same for files. In `--script` mode both return the default without prompting. `set-buffer-readonly` sets or clears the current buffer's read-only flag. `dired-list` clears and fills the current buffer with a shell-free GNU Emacs-style listing. Directories are grouped first, entries are sorted lexically, and the canonical directory path is stored as the buffer file name. `dired-entry-name` returns the file-name column from the current Dired line. See `lisp/dired.l` for a self-contained buffer-local Dired implementation; it does not require `smart-find-file.l` because it opens files with `me-find-file` after resolving them.

`ai-complete` is available in AI-enabled builds, including the default Linux and Windows builds. It sends a prompt to the configured AI service with a built-in assistant instruction, returns the generated Markdown text as a string, and returns `nil` after displaying an error. The default service is local Ollama. The optional second argument selects the

model and the optional positive third argument sets the prediction limit:

```
(ai-complete "prompt")
(ai-complete "prompt" "qwen2.5-coder:1.5b" 64)
(ai-complete-region)
(ai-complete-region "qwen2.5-coder:1.5b" 64)
```

On failure, `ai-complete` returns `nil` after displaying the underlying error. The `ai-last-error` form returns that error text as a string for scripts that need to format it themselves. An empty or whitespace-only successful response is treated as a failure. `ai-complete-region` sends the text between dot and the mark, accepts the same optional model and prediction-limit arguments, and leaves dot, the mark, and the kill ring unchanged. The AI examples use `get-region` to retain that text so both the selected prompt and the assistant response appear in the Markdown transcript.

`ai-set-endpoint` configures an HTTP endpoint and optionally sets a Bearer API key. An endpoint ending in `/v1/chat/completions` uses the OpenAI-compatible chat request and response formats. HTTPS is not supported by the compact socket client. Keep API keys in a local startup file that is not committed:

```
(ai-set-endpoint "http://127.0.0.1:11434/api/generate")
(ai-set-endpoint "http://your-gpustack-host:8090/v1/chat/completions"
                 (getenv "GPUSTACK_API_KEY"))
(ai-set-reasoning-effort "low")
(ai-set-timeout 180)
```

`ai-set-reasoning-effort` accepts `minimal`, `low`, `medium`, or `high` for OpenAI-compatible services that support the field. An empty string disables it, and configuring a new endpoint clears it. Use `low` with reasoning models that can otherwise consume the entire token limit before producing final message content. `ai-set-timeout` sets the connect/read timeout from 1 through 86400 seconds; the default is 30 seconds. The AI examples bind `ai-ask-line` to **M-A** and `ai-ask-region` to **C-X A**.

Anthropic Claude is not supported directly because the official Messages API requires HTTPS and uses different request and response formats. Claude Code and `claude.ai` URLs are not chat-completion endpoints. Use an OpenAI-compatible HTTP proxy, such as LiteLLM, when Claude access is needed:

```
(ai-set-endpoint "http://127.0.0.1:4000/v1/chat/completions"
                 (getenv "LITELLM_API_KEY"))
```

`get-char` returns the UTF-8 character under dot as a string, or `nil` at the end of a line. `get-line` returns the current line as a string without the newline; with a numeric argument, it returns a line relative to dot and returns `nil` outside the buffer. `get-region` returns the text between dot and the mark, including newline characters, as a string; it returns `nil` when no region is marked or the region is too large for the Lisp result buffer. In AI-enabled builds, `getenv` returns an environment variable as a string, or `nil` when it is not set. `count-char` returns the number of occurrences of a one-character string in another string. `insert-string` inserts a string at dot, while `insert-char` inserts one character given as a Unicode code point or a one-character string. `goto-bol`, `goto-eol`, and `forward-line` move dot without replaying keys. `save-buffer` and `kill-emacs` perform those editor actions directly. `eobp` returns `t` when dot is at the end of the buffer and `nil` otherwise. `me-find-file` opens a named file in the current logical edit context.

`define-lisp-command` registers a Lisp function as a named MicroEMACS command. Use `#'name` to pass a function reference. For compatibility, a quoted expression string is also accepted. The generated expression can be invoked with **M-X**, or bound with `define-key`, and is limited to 243 bytes. An interactive Lisp command can declare `(interactive "p")`; the command's numeric prefix is placed in its first optional parameter. For example, `M-L` runs `format-log-line` once and `M-3 M-L` runs it three times.

MicroEMACS also loads a Lisp startup file after the files named on the command line and before commands from the startup profile are executed. The default name is `.mel` on Linux and Windows. MicroEMACS looks for it in the current directory, then in the user's home directory. If no default file is found, startup continues normally. Use the **-l** option to load a specific Lisp file instead. Use the **-q** option to skip the default Lisp startup file and the default profile; explicit **-l** and **-p** files are still loaded.

Use –**script** to run a Lisp file non-interactively:

```
me --script script.l file.c
```

Script mode skips terminal setup, frame rendering, the default profile, and the default `.mel` startup file. Remaining filename arguments are opened in one headless edit context, with the last file current, before the script runs. An explicit **-l** file is loaded first; place **-l** before –**script** because the script option consumes the filename that immediately follows it. Lisp `message` output is written to standard output, bracketed diagnostics are written to standard error, and the process exits successfully only when script evaluation succeeds. Lisp scripts may begin with a `#!` shebang line, as in `examples/script.l`.

The `examples/format-code.l` script is a larger headless example. With the smart-indent library loaded, it rewrites the indentation of the current buffer:

```
cp examples/smart-indent-input.c /tmp/format-code.c
me -l lisp/smart-indent.l --script examples/format-code.l /tmp/format-code.c
```

At runtime, **C-x F** evaluates files ending in `.l`, `.lisp`, or `.mel` as Lisp source; other files are read as profiles. The explicit **M-X read-lisp** command always evaluates a Lisp file.

```
; MicroEMACS Lisp startup file
(set-variable "tab-width" 8)
(set-variable "fill-column" 72)
(define-key "C-x C-b" "buffer-menu")
(defun double (x) (* x 2))
```

The `examples/logline.l` file is the Tiny Lisp equivalent of the `examples/logline.pro` profile example. It defines a Lisp function, registers that function as the `format-log-line` command, binds it to `M-L`, formats three lines, saves the file, and quits:

```
me -l examples/logline.l filename
```

The `lisp/smart-indent.l` file is a reusable auto-indentation library. It counts unmatched braces with `count-char`, scans surrounding lines with relative `get-line` forms, and binds **Enter** to `smart-indent`:

```
me -l lisp/smart-indent.l examples/smart-indent-input.c
```

The `examples/ai-assist-ollama.l` and `examples/ai-assist-gpustack.l` files define `ai-ask-line` on **M-A**. The command sends the current line to the configured AI service and appends a Markdown transcript with `### You` and `### Assistant` headings to the `*AI*` buffer. They require an AI-enabled build; the default Linux and Windows builds include AI.

25 Display Features

MicroEMACS can display optional colors, syntax highlighting, and line numbers. These features are compiled into the current Linux and Windows builds and are designed to add little overhead to the editor's small executable size.

25.1 Colors

Color support is compiled in with the `COLOR` macro. The Linux build uses ncursesw or termcap colors, and the MinGW build uses the native Windows console API. Mode lines are rendered with a distinct color when color output is active.

When Lisp support is also enabled, configure the mode line colors in a Lisp startup file with:

```
(set-modeline-color 15 22 0 2)
```

The first two values are the foreground and background 256-color indexes; the last two are the fallback colors for 8/16-color terminals and the Windows console.

In builds with the mouse compiled in, the mouse selection highlight gets its own color the same way, in place of reverse video. The default is bright white on a deep blue background:

```
(set-selection-color 15 24 7 4)
```

Start MicroEMACS with `-C` to disable color for a session:

```
me -C filename
```

The `set-color` extended command toggles color while the editor is running. It is not bound to a key. Provide a numeric argument of zero to turn color off, or a nonzero numeric argument to turn it on.

[**unbound**] **display-visable**

Toggle a mode-line indicator that reports how much of the buffer is visible. A numeric argument sets the indicator rather than toggling it — nonzero turns it on, zero turns it off. The indicator shows **all**, **top**, **bot**, **emp**, or a percentage through the buffer.

The current implementation preserves double-width CJK characters on the Windows console, including lines that contain syntax highlighting.

25.2 Syntax Highlighting

Syntax highlighting is selected automatically from the file extension or, for executable scripts, from the interpreter named in the shebang line. The built-in definitions cover:

- C: `.c`, `.h`
- C++: `.cpp`, `.cc`, `.cxx`, `.C`, `.hpp`, `.hh`, `.hxx`, `.ipp`, `.tcc`
- Assembly: `.s`, `.S`, `.asm`
- Bash: `.sh`, `.bash`, `.bashrc`, `.bash_profile`, `.profile`, and `bash` or `sh` shebangs
- Python: `.py`, `.pyw`, `.pyi`, and `python` shebangs
- Markdown: `.md`, `.markdown`, `.mdown`, `.mkd`
- JSON: `.json`
- YAML: `.yaml`, `.yml`
- TOML: `.toml`
- CMake: `.cmake`, `CMakeLists.txt`
- Makefile: `.mk`, `.mak`, `Makefile`, `makefile`, `GNUmakefile`, and files with `Makefile` or `makefile` in the first comment line
- Dockerfile: `Dockerfile`, `Dockerfile.<variant>`, `<name>.Dockerfile`, and `.dockerfile`
- Lisp: `.lisp`, `.lsp`, `.l`, `.cl`, and `sbcl`, `clisp`, or `lisp` shebangs
- Emacs Lisp: `.el`, `.emacs`, and `emacs` shebangs
- Scheme: `.scm`, `.ss`, `.sps`, `.sls`, `.sld`, and `guile`, `gosh`, `csi`, `gsi`, or `scheme` shebangs
- SQL: `.sql`

The renderers support block comments, line comments, strings, keywords, preprocessor directives, shell variables, Python triple-quoted strings, Markdown fenced code, Lisp block comments, Scheme booleans, configuration keys, TOML tables, CMake variables and bracket comments, Makefile targets, variables, and recipes, Dockerfile instructions and build variables, Assembly labels and directives, and SQL comments. Multi-line constructs are highlighted consistently while scrolling.

Use the `c-mode`, `cpp-mode`, `assembly-mode`, `bash-mode`, `python-mode`, `markdown-mode`, `json-mode`, `yaml-mode`, `toml-mode`, `cmake-mode`, `makefile-mode`, `dockerfile-mode`, `lisp-mode`, `emacs-lisp-mode`, or `scheme-mode` extended command to select syntax for the current buffer explicitly. Use `text-mode` to explicitly disable syntax highlighting and remove the mode-line indicator. These commands are not bound to keys. SQL is selected automatically for `.sql` files. It can also be selected with **M-X sql-mode** or **M-X eval-lisp** and `(set-syntax "sql")`.

Only the lines currently visible in each window are rendered with syntax colors, and syntax state is cached to avoid redundant rescanning.

25.3 Line Numbers

Start MicroEMACS with `-n` to display Vim-style line numbers:

```
me -n filename
```

The `set-number` extended command toggles line numbers while the editor is running. Provide a numeric argument of zero to turn line numbers off, or a nonzero numeric argument to turn them on. The gutter uses a compact width based on the current line-number width and refreshes when that width changes.

26 UTF-8 and Unicode

MicroEMACS supports reading and writing text files encoded with UTF-8, a byte-oriented encoding of Unicode. UTF-8 can be thought of as a superset of ASCII: bytes less then 0x80 are identical to ASCII, and bytes greater than or equal to 0x80 are always parts of UTF-8 sequences. UTF-8 characters vary in length from one byte (ASCII) to six bytes, though four bytes is the longest typically seen.

Internally, Microemacs stores lines of text in their original UTF-8 encoding, but displays multibyte UTF-8 sequences to the user as single Unicode characters in the range 0x80 to 0xffff. Unicode characters greater then 0xffff are not supported; in practice these characters are extremely rare.

For the most part, MicroEMACS is able to display UTF-8 characters correctly, with the exception of non-spacing or combining characters. The display of these characters is undefined: sometimes they show up as modifiers of subsequent characters, or as blanks, or as a lowercase 'x' with a modifier.

On Linux, most terminal programs support a standard method for entering Unicode characters at the keyboard: hold down Ctrl and Shift, then press and release 'u', then release Ctrl and Shift, then enter the hex digits of the Unicode character followed by the Enter key. This method works when entering characters at prompts in the echo line, or while entering text in the edit buffer.

If this method doesn't work, you can still enter Unicode characters in the edit buffer using the following command:

M-C-U **unicode**

This command prompts you to enter a line of text containing the hexadecimal values of one or more Unicode characters. The hex values must not have a '0x' prefix, or any other prefix, and must be separated by spaces. MicroEMACS will then insert the corresponding UTF-8 characters into the current buffer.

As an example, entering the string `e0 e1 e2` would insert the characters `àáâ` into the buffer.

27 Building a MicroEMACS

MicroEMACS is built from portable editor sources plus a small set of operating-system and terminal sources. The top-level `Makefile` chooses `Make.Linux` on Linux and `Make.Mingw` on Windows. You can select a makefile explicitly with `make MF=Make.Linux` or `make MF=Make.Mingw`.

27.1 Operating System

MicroEMACS currently supports Linux and Windows. Support code for other operating systems has been lost (in the distant past, these included CP/M-86 and MS-DOS on the DEC Rainbow, VMS on the VAX, CP/M-68K, GEMDOS, and FlexOS V60/68K/286). The following modules contain code dependencies on the operating system:

- `sys/unix/ttyio.c` - low level UNIX terminal I/O.
- `sys/mingw/ttyio.c` - low level Windows console I/O.
- `sys/unix/spawn.c` and `sys/mingw/spawn.c` - subjob creation.
- `sys/unix/fileio.c` and `sys/mingw/fileio.c` - low level file handling.

Adding a new operating system consists mostly of adding a corresponding directory under `sys` and updating one of the makefiles.

27.2 Terminal Support

MicroEMACS supports several kinds of terminals: those supporting ncurses or termcap, and the native Windows text console (the code for real-mode PC displays and OS/2 terminal windows has been lost). The following modules contain code dependencies on the terminal type:

- `tty.c` - high-level terminal support.
- `ttyio.c` - low-level terminal support.

- `ttykbd.c` - keyboard dependencies and extensions.

Changing terminal type consists mostly of changing these files and the header file `ttydef.h`. These files are located in the `tty/ncurses`, `tty/termcap`, and `tty/mingw` directories.

Some terminals have memory mapped displays, or interfaces that act as such. These include ncurses and Windows text consoles. Support for these displays is enabled by setting the MEMMAP switch in `ttydef.h` to 1. This eliminates the fancy Gosling screen update code in `display.c`, and enables writing directly to screen memory (or to a screen buffer that the terminal interface library later writes to the screen).

To support a new memory-mapped display, you must provide a `putline` function for writing lines to the display. On old DOS-base systems, this code was written in assembly language, but on modern terminals it is written in C and placed in `tty.c`.

27.3 Building with build.sh

The `build.sh` wrapper is the recommended way to build, test, and install the editor. Run it from `tools/MicroEMACS`:

```
./build.sh -b          # Build the editor for the host system
./build.sh -b test     # Build the test targets
./build.sh -d          # Build a debug executable
./build.sh -r          # Build and strip a local release
./build.sh -m          # Build me without color, mouse, Lisp, and AI
./build.sh -a          # Build me with AI support; enabled by default
./build.sh -p          # Build a static executable in Docker
./build.sh -s          # Package the MicroEMACS source tree
./build.sh -w          # Clean and build me.exe with MinGW in Docker
./build.sh -i          # Install the Linux executable in /usr/local/bin
./build.sh -c          # Remove generated build files
```

The default builds enable color support with `-DCOLOR`, Lisp extensions with `-DLISP`, and AI completion with `-DAI`. To use termcap instead of ncursesw, pass `TERMCAP=yes` to make:

```
make MF=Make.Linux TERMCAP=yes clean all
```

AI completion is enabled by default. It requires Lisp support and links the platform socket library. Disable it with `AI=no`. The default endpoint is local Ollama, and Lisp can configure OpenAI-compatible HTTP services:

```
./build.sh -a
```

The release wrapper selects a static build automatically on Alpine and Void Linux. The publish wrapper builds a stripped static executable in the `zhoujd/alpine` Docker image.

The source package command creates `microemacs-VERSION.tar.gz` from the MicroEMACS source tree.

27.4 Windows Build

`./build.sh -w` builds a stripped `me.exe` with MinGW-w64. Outside Docker, the wrapper requires Docker and an image whose name defaults to `zhoujd/mingw:base`. Build that image from the repository root with:

```
make -C docker/dockerfiles/tool mingw-base
```

Override the image and compiler with `MINGW_IMG` and `MINGW_CC` if needed:

```
MINGW_IMG=example/mingw:base MINGW_CC=x86_64-w64-mingw32-gcc \
    ./build.sh -w
```

The Windows build also enables color support with `-DCOLOR`.

27.5 Direct Make Builds

The underlying makefiles remain available:

```
make MF=Make.Linux clean all
make MF=Make.Linux DEBUG=yes clean all
make MF=Make.Mingw CC=x86_64-w64-mingw32-gcc \
    LD=x86_64-w64-mingw32-gcc clean all
```

Tests live in `test` and can be built directly:

```
make -C test
make -C test debug
```

28 Wall Chart

Here is a list of the current key bindings in MicroEMACS. The terminal-dependent key bindings are presented at the end of this section.

Insert and Delete

Return	ins-nl
C-J	gnu-indent
C-O	ins-nl-and-backup
C-C	ins-space
C-T	twiddle
M-L	lower-word
M-U	upper-word
M-C-C	count-words
M-C	cap-word
M-D	forw-del-word
C-D	forw-del-char
Rubout	back-del-char
Backspace	back-del-char
M-Rubout	back-del-word
M-Backspace	back-del-word
C-K	kill-line
C-Y	yank
C-X C-O	del-blank-lines
M-C-U	unicode
M-Tab	set-tab-size
M-I	set-save-tabs
M-C-T	trim-line
C-X C-D	detab-line
C-X C-E	entab-line

Little Moves

C-F	forw-char
C-B	back-char
M-F	forw-word
M-B	back-word
C-N	forw-line
C-P	back-line
C-A	goto-bol
C-E	goto-eol

Big Moves

C-V	forw-page
C-Z	back-page
M-V	back-page
M-<	goto-bob
M->	goto-eob
C-X G	goto-line

Paragraphs

M-[	back-paragraph
M-]	forw-paragraph
M-J	fill-paragraph
M-C-W	kill-paragraph

Region Commands

C-@,C-space	set-mark
M-.	find-tag
M-C-.	find-tag-regexp
C-X H	mark-all
C-X C-X	swap-dot-and-mark
C-X C-L	lower-region
C-X C-U	upper-region
M-W	copy-region
C-W	kill-region
M-+	indent-region

Rectangles

C-X r C-@	rectangle-anchor
C-X r C-c	clear-rectangle
C-X r C-o	open-rectangle
C-X r C-t	string-rectangle
C-X r C-w	kill-rectangle
C-X r M-W	copy-rectangle
C-X r C-y	yank-rectangle

Search and Replace

C-S	forw-i-search
M-S	forw-i-search
C-R	back-i-search
M-C-F	fold-case
M-P	search-paren
M-Q	query-replace
M-%	query-replace
M-R	replace-string
C-S RET	forw-search
C-R RET	back-search
M-C-S	forw-regexp-search
M-C-R	back-regexp-search
M-/	reg-replace
M-?	reg-query-replace
C-X %	replace-region
M-#	spell-word

File and System Operations

C-X C-I	file-insert
C-X C-V	file-visit
C-X C-R	file-read
C-X C-W	file-write
M-T	file-save
C-X C-S	file-save
C-X S	file-save-all
C-X C-F	find-file
C-X C-H	hexl-find-file
M-@	set-file-name
M-$	set-newline
C-X B	use-buffer
C-X K	kill-buffer
M-~	umark-buffer
C-L	refresh
C-Q	quote
C-X Q	quote
M-X	extended-command
C-C	spawn-cli
C-X C-G	abort
C-G	abort
M-C-G	abort
C-X C-C	quit
C-X C	spawn-cli
M-Z	quick-quit
C-X !	shell-command
C-X @	pipe-command
C-X #	filter-buffer
C-X $	change-dir
C-X D	dired
C-X I	spell-region

Windows

C-X 1	only-window
C-X 2	split-window
C-X 0	zap-window
C-X +	balance-windows
M-!	reposition-window
C-X Z	enlarge-window
C-X C-Z	shrink-window
C-X N	forw-window
C-X O	forw-window
C-X P	back-window
C-X C-P	up-window
C-X C-N	down-window
M-C-V	scroll-next-down
M-C-Z	scroll-next-up
C-X V	display-version
C-X =	display-position
C-X C-B	buffer-menu
C-X C-Q	toggle-readonly

Macros and Profiles

C-X (	start-macro
C-X)	end-macro
C-X E	execute-macro
C-X F	read-profile

Tags and Errors

M-,	find-cscope
M-G	find-grep
M-.	find-tag
M-C-.	find-tag-regexp
C-X 4 .	find-tag-other-window
M-C-E	goto-error
C-X ‘	next-error

PC Function Keys

F1	help
F2	file-save
F3	file-visit
F4	quit
F5	undo
F6	display-buffers
F7	redo
F8	forw-buffer
F9	search-again
F10	only-window
F11	find-cscope
F12	next-cscope
Up	back-line
Down	forw-line
Left	back-char
Right	forw-char
PgUp	back-page
PgDn	forw-page
Home	goto-bol
End	goto-eol
C-Left	back-word
C-Right	forw-word
C-PgDn	down-window
C-PgUp	up-window
C-Home	goto-bob
C-End	goto-eob
Insert	yank
Delete	forw-del-char

Zenith Z-19/29 Keypad

Pad-1	search-again	**Pad-Enter**	ins-nl	**Blue-9**	yank
Pad-2	forw-line	**Blue-0**	ins-nl-and-backup	**Blue-Period**	abort
Pad-4	back-char	**Blue-2**	goto-eob	**Blue-Enter**	ins-nl
Pad-5	forw-search	**Blue-4**	goto-bol	**M-A**	back-page
Pad-6	forw-char	**Blue-5**	query-replace	**M-B**	forw-page
Pad-7	forw-del-word	**Blue-6**	goto-eol	**M-C**	forw-word
Pad-8	back-line	**Blue-7**	yank	**M-D**	back-word
Pad-9	kill-line	**Blue-8**	goto-bob		

Index

-l, 10
-p, 10

Abort, 8
ALT key, 6
Argument, 8, 13, 30
autocompletion, 8

back-buffer, 25
back-char, 12
back-del-char, 15
back-del-word, 17
back-i-search, 20
back-line, 12
back-page, 12
back-paragraph, 18
back-search, 19
back-window, 26
back-word, 17
Backspace, 15
Backup file, 9, 22
balance-windows, 27
bash, 11
Binary files, 23
bind-to-key, 28
borland-indent, 13
Buffer, 5, 10, 13, 21, 24, 25
Buffer changed flag, 5, 22, 24
Buffer list, 25
Buffer Management, 24
Buffer name, 5, 24
Buffer, reverting, 22
buffer-menu, 25
Building a MicroEMACS, 37
Byte order, in a hexl buffer, 23

C-, 11, 16
C-A, 11
C-B, 12
C-C, 13
C-D, 15
C-E, 12
C-F, 12
C-G, 16
C-H, 15
C-I, 13
C-J, 13, 14
C-K, 15
C-L, 26
C-M, 13
C-N, 12
C-O, 14
C-P, 12
C-Q, 14
C-R, 19, 20
C-S, 19, 20
c-shell, 11
C-space, 11
C-T, 14
C-V, 12
C-W, 11, 15
C-w, 16
C-X , 11, 23
C-X (, 24
C-X), 24
C-X +, 27
C-X 0, 26
C-X 1, 26
C-X 2, 26
C-X 4 ., 28
C-X =, 13
C-X #, 23
C-X $, 23
C-X B, 25, 26
C-X C, 11
C-X C-B, 25
C-X C-C, 10
C-X C-D, 14
C-X C-E, 14
C-X C-F, 21
C-X C-H, 23
C-X C-I, 22
C-X C-L, 18
C-X C-N, 26
C-X C-O, 16
C-X C-P, 26
C-X C-Q, 25
C-X C-R, 22, 26
C-X C-S, 22
C-X C-U, 18
C-X C-V, 22, 26
C-X C-W, 22, 28
C-X C-X, 11
C-X C-Z, 26
C-X D, 23
C-X E, 24
C-X F, 31
C-X G, 12
C-X H, 11
C-X I, 21
C-X K, 25
C-X N, 26, 28
C-X O, 26
C-X P, 26
C-X Q, 14
C-X r, 16
C-X r C-, 16
C-X r C-c, 16
C-X r C-o, 16
C-X r C-t, 16

C-X r C-w, 16
C-X r C-y, 16
C-X r M-W, 16
C-X S, 22
C-X V, 27
C-X Z, 26
C-Y, 15
C-Z, 12
cap-word, 17
Carriage return, 13, 19
Case Conversion, 18
Case folding, 19
change-dir, 23
clear-mark, 11
clear-rectangle, 16
Color, 35
Colors, 35
Command file, 29
Command interpreter, 11
CONTROL characters, 6
CONTROL flag, 5
Control-Q, 10, 14, 30
Control-S, 10, 19, 22
Control-Z, 10
copy-rectangle, 16
copy-region, 15
count-words, 17
CP/M, 11, 19, 30
cscope, 9, 28
ctags, 28
CTLX flag, 5
Cursor keys, 6

DEC, 4
Del, 15
del-blank-lines, 16
Deleting, 15
detab-line, 14
Directory, 23
dired, 23
Display features, 35
display-bindings, 28, 30
display-buffers, 25
Display-cell grid, 16
display-message, 27
display-position, 13
display-version, 27
display-visable, 36
Dot, 8, 11, 13, 24, 25
down-window, 26

echo, 32
Echo line, 5, 7, 13, 27, 32
EDT, 6
EDT.PRO, 6
EMACS, 4
end-macro, 24
enlarge-window, 26
entab-line, 14
Error navigation, 29
ESC key, 5, 6
ESC X, 24, 27
ESC X, 31
eval-lisp, 32
execute-macro, 24
Extended command, 7, 24, 27
extended command, 30

File, 5
File name, 5, 10, 21, 24
File, finding, 21
File, hexl, 23
File, inserting, 22
File, reading, 22, 29
File, saving, 22
File, visiting, 22
File, writing, 22
file-insert, 22
file-read, 22
file-save, 22, 31
file-save-all, 22, 23
file-visit, 22
file-write, 22
Files, 21
Fill column, 18
fill-paragraph, 18
filter-buffer, 23
find-cscope, 28
find-file, 21
find-grep, 28
find-tag, 28
find-tag-other-window, 28
find-tag-regexp, 28
FlexOS, 6, 9, 11, 19, 30
fold-case, 19
forw-buffer, 25
forw-char, 12
forw-del-char, 15
forw-del-word, 17
forw-i-search, 20
forw-line, 12
forw-page, 12
forw-paragraph, 18
forw-search, 19
forw-window, 26
forw-word, 17
Function keys, 6

gnu-indent, 14
goto-bob, 12
goto-bol, 11
goto-eob, 12
goto-eol, 12
goto-error, 29

goto-line, 12
Group size, in a hexl buffer, 23

help, 28, 31
Hexl editing, 23
hexl-endian, 23
hexl-find-file, 23
hexl-group-size, 23
hexl-packed, 23

Incremental search, 20
Indent, 13, 14
indent-region, 15
ins-macro, 24
ins-nl, 13
ins-nl-and-backup, 14
ins-nl-and-indent, 13
ins-self, 7, 13
ins-self-with-wrap, 13, 18
ins-space, 13
Inserting, 13
Inserting a file, 22

jeff-exit, 11
just-one-space, 15

Key binding, 7, 27, 29
Key Binding Commands, 27
Keyboard conventions, 5
Keyboard Macros, 24
Kill buffer, 15
kill-buffer, 25
kill-line, 15
kill-paragraph, 18
kill-rectangle, 16
kill-region, 15
Killing, 15

Line endings, 23
Line numbers, 35, 36
Linux, 37
Lisp Extensions, 32
Literal character, 14
LK-201, 7
lower-region, 18
lower-word, 17

M-, 21, 27
M-+, 15
M-/, 21
M-<, 12
M->, 12
M-?, 21
M-[, 18
M-#, 21
M-$, 23
M-], 18
M-B, 17
M-C, 17
M-C-., 28
M-C-C, 17
M-C-F, 19
M-C-H, 17
M-C-T, 14
M-C-U, 37
M-C-V, 27
M-C-W, 18
M-C-Z, 27
M-D, 17
M-F, 17
M-I, 14
M-J, 18
M-L, 17
M-P, 20
M-Q, 20
M-R, 21
M-Rubout, 17
M-T, 22
M-Tab, 14
M-U, 17
M-V, 12
M-W, 15, 16
M-X, 27
M-Z, 10
M- , 25
Macro, 24, 29
Mark, 8, 11, 24, 25
mark ring, 11
mark-all, 11
Messages, 7, 27
META flag, 5
MINCE, 4, 18
Mode, 5
Mode line, 5
mode line, 25
Moving Around, 11

name-macro, 24
ncursesw, 38
next-cscope, 28
next-error, 29

only-window, 26
Open line, 14
open-rectangle, 16
Operating System, 37

Packed groups, in a hexl buffer, 23
Pad-1, 20
Paragraphs, 17
PC, 6
PC-DOS, 6, 9, 11, 19, 30
pipe-command, 23
pop-tag-mark, 29
Profile, 6, 10, 24, 29
profile, 10

Profiles, 29
Prompt, 7

query-replace, 20
quick-quit, 10
Quit, 10, 23
quit, 31
Quitting, 10
quote, 14, 30

read-only, 10, 25
read-profile, 31
Reading a file, 22, 29
Rectangle, 16
rectangle-anchor, 16
Rectangles, 16
redo, 29
refresh, 26
reg-query-replace, 21
reg-replace, 21
Region, 8, 11, 15, 18, 21
regular expressons, 19
replace-region, 21
replace-string, 21
reposition-window, 27
Return, 13
revert-buffer, 22
Rubout, 15

Saving a file, 22
Screen layout, 5
scroll-next-down, 27
scroll-next-up, 27
search-again, 20
search-paren, 20
Searching, 19
set-color, 36
set-file-name, 21
set-fill-column, 18
set-mark, 11
set-newline, 23
set-number, 36
set-save-tabs, 14
set-tab-size, 14
Shell, 11, 23
shell-command, 11
Shift-5, 20
shrink-window, 26
spawn-cli, 11
spell-region, 21
spell-word, 21
split-window, 26
start-macro, 24
Starting, 8, 29
string-rectangle, 16
Subjob, 11
swap-dot-and-mark, 11
Syntax highlighting, 35, 36

Tab, 13
TAB, in a hexl buffer, 23
tags, 28
termcap, 38
Terminal Support, 37
toggle-readonly, 10, 25
trim-line, 14
twiddle, 14

umark-buffer, 25
Undo, 23, 29
undo, 29
Unicode, 37
unicode (command), 37
UNIX, 9, 11, 30
up-window, 26
upper-region, 18
upper-word, 17
use-buffer, 25
UTF-8, 37

Version, 27
Visiting a file, 22
VME/10, 6
VMS, 6, 9, 11, 30
vmware-indent, 14
VT-100, 6, 7, 18

Wall chart, 40
Window, 5, 10, 25, 28
Window Management, 25
Word wrap, 13, 18
Words, 17
Writing a file, 22

XON/XOFF, 10, 14, 19, 22

yank, 15
yank-rectangle, 16

zap-window, 26
Zenith Z-19, 6
Zenith Z-29, 6, 10, 19, 20, 22